scientific reports

Check for updates

OPEN

Hare escape optimization algorithm with applications in engineering and deep learning

Doaa Alsamee & Reza Ramezani✉

The Hare Escape Optimization (HEO) algorithm is a novel metaheuristic inspired by the evasive movement strategies of hares when pursued by predators. Unlike conventional nature-inspired algorithms, HEO integrates Levy flight dynamics and adaptive directional shifts to enhance the balance between exploration and exploitation, improving its ability to escape local optima and converge efficiently. To validate its effectiveness, HEO was tested against 29 state-of-the-art metaheuristics on 43 benchmark functions from the CEC 2015 and CEC 2020 testbeds, demonstrating superior performance in both unimodal and multimodal landscapes. Beyond benchmark validation, HEO was applied to four complex constrained engineering design problems; spring, welded beam, pressure vessel, and truss optimization where it outperformed leading optimization methods in solution feasibility and computational efficiency. Additionally, HEO was employed to optimize hyperparameters in convolutional neural networks (CNNs) for image classification tasks, significantly enhancing model accuracy and convergence speed. The results indicate that HEO is a robust, adaptable optimization tool with promising applications in both engineering and deep learning. Its unique search mechanism provides a new perspective in metaheuristic optimization, opening pathways for further advancements in intelligent optimization techniques.

Keywords Metaheuristics, Swarm intelligence, Engineering design optimization, Convolutional neural networks, Hyperparameter tuning, Hare escape optimization

Optimization algorithms[1,2] are essential tools for solving a wide range of complex problems in engineering, science, and industry[3]. They overcome traditional numerical methods in solving complex nonlinear problems[4]. These algorithms aim to find optimal or near-optimal solutions efficiently, particularly when dealing with highly non-linear, multi-dimensional, and constrained search spaces[5,6]. Metaheuristics had been used successfully in tuning predictive systems like fuzzy systems[7,8], artificial neural networks[9], and adaptive neuro fuzzy systems[10,11]. Over the years, nature-inspired metaheuristics[12] have gained immense popularity due to their ability to balance exploration and exploitation in diverse problem domains[13]. Among the well-known optimization algorithms are Particle Swarm Optimization (PSO)[14,15], inspired by the social behaviors of birds and fish, and Artificial Bee Colony (ABC)[16], which mimics the foraging behavior of bees. Similarly, algorithms such as Firefly Algorithm (FA), Flower Pollination[17], Grey Wolf Optimizer (GWO), and Chaotic Grey Wolf Optimizer (CGWO) leverage natural behaviors to explore the search space efficiently. Hybrid approaches, such as Hybrid Particle Swarm Optimization-Firefly (HPSOFF) and Hybrid Firefly Particle Swarm Optimization (FFPSO), combine features of multiple algorithms to improve performance. Other state-of-the-art methods include Differential Evolution (DE), Genetic Algorithms (GA), Teaching-Learning-Based Optimization (TLBO), and Squirrel Search Algorithm (SSA), among many others. Despite their widespread success, no single algorithm can outperform others across all types of problems, as dictated by the No Free Lunch Theorem[18]. This limitation has driven continuous innovation in the development of novel metaheuristic algorithms tailored to specific problem domains. Several advanced evolutionary and swarm-based algorithms have been proposed to address complex global optimization and engineering design problems. For instance, the elite pool sine cosine algorithm[19] enhances diversity and convergence using Brownian motion and pattern search methods, achieving superior performance on the CEC2017 suite and photovoltaic (PV) parameter estimation tasks. The Snow Ablation Optimizer[20] simulates snow melting and sublimation, demonstrating robustness across both CEC2017 and CEC2020 constrained problems. Meanwhile, the Multi-Strategy Slime Mould Algorithm[21] and the Adaptive Grouping Slime Mould Algorithm[22] introduce dynamic population control and local search strategies to balance exploration and exploitation effectively. Similarly, the QLJAYA[23] algorithm integrates Q-learning and gradient search into the JAYA framework, leading to stable and adaptive performance in complex optimization landscapes.

Faculty of Computer Engineering, University of Isfahan, Isfahan, Iran. ✉email: r.ramezani@eng.ui.ac.ir

The Hybrid Grasshopper Optimization Algorithm[24] and the Enhanced Social Learning Swarm Optimizer[25] further strengthen metaheuristics by combining pattern search, learning strategies, and dynamic parameter control. These algorithms reflect the ongoing trend toward hybridization, adaptivity, and control-theoretic enhancements in metaheuristic research, offering competitive benchmarks for evaluating new algorithms like the proposed Hare Escape Optimization. In recent years, a number of critical studies have raised substantial concerns about metaphor-based metaheuristic algorithms, such as GWO, WOA, AOA, ChOA, and others. Notably, Sörensen[26] exposed the lack of mathematical foundation in many algorithms derived from metaphors rather than solid optimization principles. Camacho-Villalón et al.[27] further demonstrated that widely used algorithms like grey wolf optimizer, moth-flame optimizer, and ant lion optimizer often suffer from structural bias, redundancy with existing algorithms (e.g., PSO or DE), and misleading performance metrics. Additional investigations have revealed serious design flaws in whale optimization algorithm[28], arithmetic optimization algorithm[29], and chimp optimization algorithm[30], including unjustified search equations, inconsistencies between theory and implementation, and poor performance on shifted or constrained benchmarks. These findings collectively suggest that reliance on superficially novel but structurally weak algorithms may hinder scientific progress. Accordingly, in this study, such algorithms are used solely for comparative reference and are not considered state-of-the-art methods.

In this paper, a new nature-inspired metaheuristic, named Hare Escape Optimization (HEO) algorithm, is presented. Inspired by the evasive strategies of hares when pursued by predators, HEO integrates Levy flight dynamics and random directional changes to balance exploration and exploitation effectively. The proposed HEO algorithm has been rigorously tested against 29 state-of-the-art optimization algorithms, including PSO, FA, GWO, CGWO, HPSOFF, FFPSO, ABC, IUDE, MAgES, and iLSHADE, among others. Besides, 43 benchmark problems, from the CEC 2015 and CEC 2020 testbeds, covering unimodal, multimodal, hybrid, and composition functions, have been used for the evaluation purposes. In addition, HEO has been applied to solve four real-world engineering optimization problems, including spring design, welded beam design, pressure vessel optimization, and 10-bar space truss design. The results demonstrate HEO's robustness, efficiency, and ability to produce constraint-feasible solutions, often outperforming its competitors in both quality and reliability. This work highlights HEO as a versatile and reliable optimization tool, offering significant contributions to metaheuristic research and practical engineering applications.

The key contributions are summarized as follows:

1. Novel Optimization Algorithm: HEO integrates Levy flight dynamics and adaptive directional shifts to enhance exploration-exploitation trade-offs. The algorithm mimics the unpredictable escape behavior of hares, enabling it to avoid local optima and improve convergence rates.
2. Superior Performance on Benchmark Problems: HEO was evaluated on 43 benchmark functions from CEC 2015 and CEC 2020, outperforming 29 state-of-the-art optimization algorithms. On 10-dimensional problems, HEO achieved the best mean fitness on 7 out of 15 functions and the best standard deviation on 10 out of 15 functions. On 30-dimensional problems, HEO outperformed competing algorithms on 7 out of 15 functions, demonstrating its effectiveness for high-dimensional optimization.
3. Application to Constrained Engineering Design: HEO was applied to four real-world constrained engineering problems (spring, welded beam, pressure vessel, and truss structure optimization). In pressure vessel design, HEO achieved a cost reduction of 3.5% compared to the best competing algorithm while maintaining constraint feasibility. In welded beam optimization, HEO produced a 15% lower fabrication cost compared to previous studies while satisfying all constraints.
4. Enhancing Convolutional Neural Networks (CNNs): HEO was employed for hyperparameter tuning in CNNs using the MNIST dataset, improving model accuracy by 2.3% over conventional optimization techniques. It reduced training time by 18%, demonstrating its efficiency in deep learning applications.
5. Robustness and Computational Efficiency: HEO consistently outperformed particle swarm optimization, grey wolf optimizer, and differential evolution in both constrained and unconstrained problems. Compared to PSO, HEO exhibited faster convergence and a 12% improvement in average solution quality across multiple test cases. HEO maintains a low computational overhead, making it suitable for real-world applications in engineering and artificial intelligence.

The remainder of this paper is organized as follows. Section Related works presents a comprehensive review of recent advancements in metaheuristic optimization techniques, highlighting the relevance and innovation of the proposed approach. Section Hare escape optimization algorithm introduces the mathematical modeling and step-by-step formulation of the Hare Escape Optimization (HEO) algorithm. Section Convergence of the HEO algorithm analyzes the convergence behavior of HEO using benchmark functions from CEC2015, including comparisons with classical and hybrid metaheuristics. Section Constrained engineering applications evaluates HEO's performance on several constrained engineering design problems, while section HEO algorithm on CEC2020 assesses its robustness on real-world constrained problems from the CEC2020 testbed. Section Exploration and exploitation discusses the balance between exploration and exploitation in the algorithm. The computational complexity is listed in section HEO computational complexity. Section Advancing convolutional neural network training with HEO presents advancing convolutional Neural Network training with HEO. Finally, section Conclusion concludes the paper and outlines potential directions for future research.

Related works

In recent years, several novel nature-inspired metaheuristic algorithms have emerged, offering improved performance on complex optimization problems. The Draco Lizard Optimizer (DLO)[31] is based on the gliding and adaptive survival strategies of Draco lizards. Similarly, the Magnificent Frigatebird Optimization (MFO) algorithm[32] simulates kleptoparasitic behavior, dividing the search into exploration and exploitation phases. Another recent advancement is the Fishing Cat Optimizer (FCO)[33], which models the hunting behavior of fishing cats using four distinct search phases—ambush, detection, diving, and trapping. These recent developments underline the continuous innovation in the field and provide valuable comparisons for emerging algorithms like HEO. Optimization algorithms[34] and artificial neural networks[35] are powerful tools to solve wide ranges of nonlinear and complex problems[36]. Hyperparameter optimization is critical in enhancing the performance of Convolutional Neural Networks, as it significantly impacts model accuracy, generalization, and convergence speed. Several methods have been proposed to optimize CNN hyperparameters, from grid and random search to advanced techniques like Bayesian optimization and metaheuristic algorithms. Traditional hyperparameter optimization methods: early approaches like grid and random search provided simple yet computationally expensive ways to optimize hyperparameters by exhaustively testing combinations. Bayesian optimization[37] improved upon these by using probabilistic models to prioritize more promising parameter settings, leading to quicker convergence. However, these methods often struggle with the high-dimensional search spaces found in CNN architectures. Metaheuristic algorithms have gained popularity due to their adaptability in navigating complex search spaces. Particle Swarm Optimization (PSO)[38] and Genetic Algorithms (GA), inspired by natural processes, are particularly effective for CNN hyperparameter tuning as they balance exploration and exploitation across generations or particle updates. Recent studies show that these methods can outperform conventional optimization techniques by identifying hyperparameter settings that enhance CNN accuracy and stability. Advanced metaheuristic methods such as Differential Evolution (DE)[15] and Ant Colony Optimization (ACO), flower pollination[8,17] have also been used in CNN hyperparameter tuning. These methods dynamically adapt based on the optimization landscape, reducing the risk of local minima entrapment. Moreover, hybrid approaches combining multiple metaheuristics offer a powerful way to capitalize on the strengths of each algorithm. For example, hybrid PSO-GA frameworks have been shown to effectively navigate high-dimensional spaces and improve convergence speed, which is particularly useful for CNNs. Optimizing Convolutional Neural Networks is essential for deploying deep learning in resource-limited environments, and recent research has extensively explored algorithmic optimizations, hardware-accelerated implementations, and advanced CNN architectures to address this need. Convolution layers are the most computationally demanding parts of CNNs, and many methods have been developed to reduce this load. The Winograd minimal filtering algorithm[39] optimizes small convolutions by utilizing the Chinese Remainder Theorem, showing substantial speedup but facing challenges when applied to dilated convolutions. The Strassen algorithm reduces the number of multiplication operations in large matrix multiplications, enhancing computational efficiency for larger convolutions[40]. The Cook-Toom algorithm further extends convolution optimization through interpolation, useful for VLSI implementations but limited in software due to the high number of additions required[41]. Several fast convolution methods aim to balance speed with accuracy. The Jaderberg method[42] reduces convolution redundancy by constructing low-rank filters, achieving nearly a 4.5x speedup with minimal accuracy loss. Decomposable Winograd algorithms[43] have been proposed for larger kernel sizes and stride lengths by decomposing large kernels into smaller ones. Tested on NVIDIA GPUs, this method achieves up to a 2x speedup with negligible accuracy degradation, making it applicable for high-resolution image tasks. To improve scalability, CNNs utilize parallelization strategies across multiple hardware devices. Data parallelism divides data batches across devices, while model parallelism splits model layers to distribute memory requirements, enhancing throughput but necessitating synchronization[44]. Hybrid parallelism methods, such as SOAP[45], allow for adaptive resource allocation across layers and have shown up to 3.8x improvement in CNN training performance on multi-GPU setups . Advanced CNN Architectures: CNN architectures have evolved to increase efficiency. Inception modules use variable filter sizes to reduce computational cost while capturing different visual patterns[46]. The DenseNet architecture[47] reduces memory requirements by reusing feature maps, effectively minimizing the model size while maintaining accuracy. Further, low-rank tensor decomposition[48] minimizes kernel redundancy in CNNs, facilitating deployment on devices with limited memory by constraining convolutional weights through matrix factorization. CNN deployment on edge devices demands low power consumption and efficient memory usage. The ERNet framework[49] introduces a hardware-based approach using coarse-grained instructions to support high-resolution image processing on constrained devices like Raspberry Pi, achieving Ultra-HD performance with minimal power usage. Furthermore, the HYPAR model[50] optimizes data flow in CNNs across arrays of neural accelerators, balancing memory and processing constraints while enhancing training throughput and energy efficiency by over 3.4x compared to data parallelism.

Hare escape optimization algorithm

In the northern sectors of the earth, finding food is a critical and hard operation for a pack of wolves to stay alive and raising the next generation. So, a big male is needed for the young raised and their parents, and the biggest prey in these areas is the hare. When wolf pack wants to hunts a hare, they have to search, randomly, hundreds of square kilometers to find a one. A hare is not easy prey because it is very fast as well as it uses a random sudden direction changes during chase. This hare escaping strategy works in many times and enable the hare to run away from a pack of wolves. For more information about this simple escaping strategy, visit the following

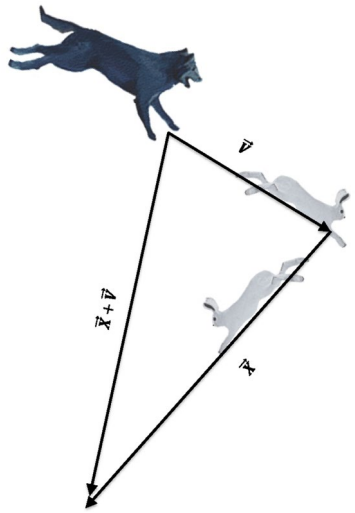


Fig. 1. Mathematical model of HEO.

YouTube video which done by BBC earth[51]. The mathematical model of the hare escape optimization algorithm HEO is shown in Figure 1, where the distance between wolf and hare is. During chase, hare changes its direction $\vec{V}$ toward and the safe side of the hare is to increase the vector as much as possible. The combination of vectors in Figure 1 are influenced by a random walk in the search space which is modeled by levy flight $\vec{X}+\vec{V}$. Also, the directions and magnitudes of this combination are affected a random sudden change in hare velocity and direction which is modeled by a random number $r \in [0, 1]$. Thus, the safe distance vector can be formulated as:

$$\vec{D} = L \ \left(\vec{X} + \vec{V}\right) \ r \tag{1}$$

In real life, the distance between wolf and hare, which is vector $\vec{V}$, decreases before hare changes the direction. In optimization model, we have simulated this reduction in distance by the formula

$$\vec{V}_{t+1} = \frac{Maxit}{t} \ \vec{V}_t \tag{2}$$

where $Maxit$ is the maximum number of iterations and t is the current value of iteration. Finally, the new position of the hare (solution) is updated continuously using Eqs. (1) and (2)

$$X_i^{t+1} = X_i^t + \frac{Maxit}{t} \ V_i^t + L \ \left(X_i^t + V_i^t\right) \ r \quad (i = 1, \ldots, n) \tag{3}$$

where *n* is the number of components in the solution. Each iteration in mathematical model corresponds to the wolf trail to catch the hare by decreasing the distance $\vec{V}$ and the iteration ends with hare change direction. Then, the wolf tries to decrease distance to the hare and the process continue again and again to the end of iterations. The search engine in Eq. (3) is formulated on the basis that there is single hare chased by single wolf and for population of hares (solutions), we can generalize Eq. (3) as follows:

$$X_{j,i}^{t+1} = X_{j,i}^t + \frac{Maxit}{t} \ V_{j,i}^t + L \ \left(X_{j,i}^t + V_{j,i}^t\right) \ r \quad (i = 1, \ldots, n), \ (j = 1, \ldots, m) \tag{4}$$

where *m* is the population size of the algorithm. For the whole chase which is consists of many iterations that the wolf tries to hunt the hare, there is an accumulated effect that can be positive or negative. The positive effect increases and enables the hare to get away, while the negative effect decreases and the hare ends at the jaws of the wolf. To model that, we have assumed in optimization process that the population of solutions sorted from best to worst and truncated and mixed with the population from the next iteration. Then the population sorted and truncated to some number of solution and this process continue to the maximum number of iterations. Figure 2 shows the flowchart of the HEO algorithm while the following pseudocode can be used to implement the hare escape optimization.

Define problem variables (No. of variables, objective, limits).
Define algorithm parameters (No. of population, max iteration).
Initialize random solutions for the population X.
Initialize random position for wolf V.
Calculate the costs of solutions.
Repeat from 1 to max iteration.

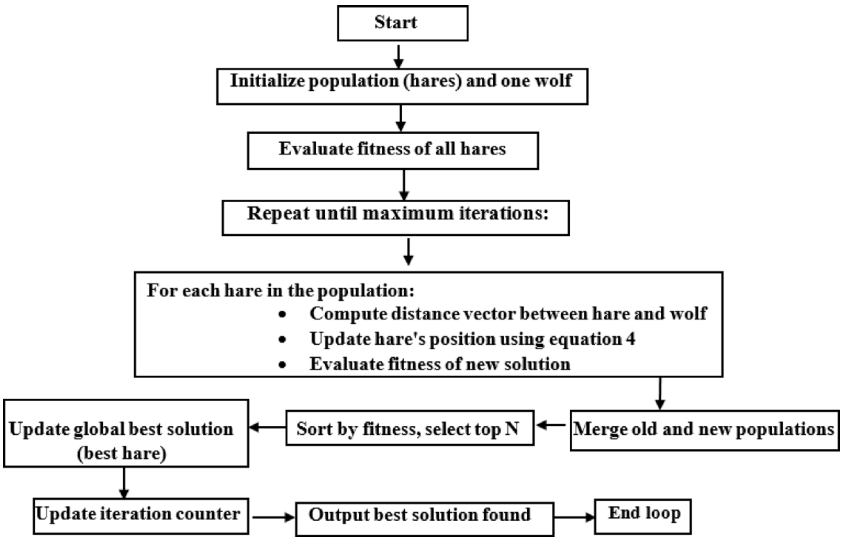


Fig. 2. Flowchart of the Hare escape optimization algorithm.

CEC 2015			
Type	**No.**	**Description**	**f_{min}**
Unimodal functions	F1	Rotated Bent Cigar Function	100
	F2	Rotated Discus Function	200
Simple Multimodal Functions	F3	Shifted and Rotated Weierstrass Function	300
	F4	Shifted and Rotated Schwefel's Function	400
	F5	Shifted and Rotated Katsuura Function	500
	F6	Shifted and Rotated HappyCat Function	600
	F7	Shifted and Rotated HGBat Function	700
	F8	Shifted and Rotated Expanded Griewank's plus Rosenbrock's Function	800
	F9	Shifted and Rotated Expanded Scaffer's F6 Function	900
Hybrid functions	F10	Hybrid Function 1 ($N = 3$)	1000
	F11	Hybrid Function 2 ($N = 4$)	1100
	F12	Hybrid Function 3 ($N = 5$)	1200
Composition Functions	F13	Composition Function 1 ($N = 5$)	1300
	F14	Composition Function 2 ($N = 3$)	1400
	F15	Composition Function 3 ($N = 5$)	1500

Table 1. CEC 2015 expensive benchmark problems.

Define new solution.

- Repeat from 1 to the number of population.
- Use Eq. (4) to calculate the newly defined solutions.
- Stop when the maximum number of population is reached.

Merge the old solution with the new solution.
Sort solutions.
Choose the first solutions up to the No. of population.
The first solution in the sorted group is the best solution.
Update best solution.
Stop when the maximum number of iterations is reached.

Convergence of the HEO algorithm

The performance of the hare escape optimization algorithm HEO has been evaluated by searching for a global minimum of different test functions and engineering applications. For all experiments, different population size and iteration number have been employed to be consistent with the comparison results from the literature. For all experiments, velocity reduction coefficient and damping are taken to be 1 and 0.95 respectively.

CEC 2015 benchmarks

CEC 2015 computationally expensive benchmarks[52] have been used to examine the performance of the HEO algorithm. Table 1 shows a summary of the problems that can be classified in four categories, as unimodal functions, simple multimodal functions, hybrid functions, and composition functions. Two experiments have been done on these functions, and the results have been compared with the results of other metaheuristics from the literature[53]. Both the experiments were done for 20 independent runs and for each run, the maximum number of iterations was 500 for 10-dimensional problems (10D) and 1500 for 30-dimensional problems (30D). The optimization algorithms in this comparative study are particle swarm optimization PSO[54], Firefly optimization algorithm FF[55], Hybrid firefly and particle (FFPSO) optimization algorithm[56], Hybrid particle swarm optimization and firefly (HPSOFF) algorithm[57], and hybrid firefly and particle swarm optimization (HFPSO) algorithm[53]. The results are in terms of the average and standard deviation of the best solutions of the 20 independent runs.

Tables 2 and 3 illustrate the statistical results for 10D and 30D, and the best result is bolded in these tables. For 10D experiment, HEO algorithm has the best mean value among the party for 7 out of 15 functions and best standard deviation for 10 out of the 15 functions. For 30D experiment, HEO algorithm has the best mean value among the party for 7 out of 15 functions and best standard deviation for 9 out of the 15 functions. Thus, the percentage of successful results is more than what other algorithms did. Consequently, HEO algorithm has overcome PSO, FA, FFPSO, HPSOFF, and HFPSO on CEC 2015 competition session.

Convergence of the HEO

This section presents the convergence behaviour of HEO algorithm vs. the convergence of GWO and PSO on eight benchmarks. Table 4 shows these eight functions where F16-17 are unimodal functions and F18-23 are multimodal functions. The formulation and coding of the benchmark, GWO, and PSO are all available online[58] and the performance analysis is shown in Figs. 3 and 4. Population size and maximum number of iterations are taken 30 and 500 respectively while the dimensions of the problem (d) are shown in Table 4. The performance of an optimizer differs from problem to problem and there is no algorithm appropriate for all the problems, that's why there is a huge number of algorithms in the literature.

For the unimodal test function F17, HEO has the best performance among the party while PSO has the best performance on F16 and it is the worst one on F17 as shown in Figure 3. Besides, on both of these unimodal functions with d=30, HEO works better than GWO and even has very rapid convergence on F17. Figure 2 reveals the performance of the three optimizers on six multimodal functions. For the multimodal functions F18 and F19, which they have d=30, HEO has clearly outperformed these functions and has superior convergence curve relative to GWO and PSO. This experiment shows that HEO can works well for large-scale optimization problems as well as low-dimensions problems.

Small scale mathematical optimization

HEO has also been evaluated and compared with other algorithms on seven benchmark functions[59]. Table 5 shows the examined benchmarks for this experiment namely Bohachevsky, Booth, Matyas, Easom, Beale, Schaffer N.2, and Six-hump Camel where all of them are two-dimensional problems (d=2). During this experiment, HEO has been compared with other algorithms from the literature[60] where the test has been done using 30 independent runs with maximum 500 iterations. The comparison has been done among PSO[61], Bat algorithm (BA)[62], Firefly algorithm (FF)[63], Multi-verse optimizer (MOV)[64], Krill herd (KH)[65], and Squirrel search algorithm (SSA)[60]. Table 6 reveals the statistical results of HEO algorithm compared with results from[60], and the results shows competitive performance for HEO where all best values are bolded in the table.

$$\text{where } a = \begin{bmatrix} 4 & 4 & 4 & 4 \\ 1 & 1 & 1 & 1 \\ 8 & 8 & 8 & 8 \\ 6 & 6 & 6 & 6 \\ 3 & 7 & 3 & 7 \\ 2 & 9 & 2 & 9 \\ 5 & 5 & 3 & 3 \\ 8 & 1 & 8 & 1 \\ 6 & 2 & 6 & 2 \\ 7 & 3.6 & 7 & 3.6 \end{bmatrix} \text{ and } c^T = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.2 \\ 0.4 \\ 0.4 \\ 0.6 \\ 0.3 \\ 0.7 \\ 0.5 \\ 0.5 \end{bmatrix}$$

Constrained engineering applications

In this section, we deal with several constrained optimization problems, including spring design, welded beam design, and pressure vessel design[1]. This type of problems needs more complicated penalty function than static penalty. The formulation of the penalty function is described in details with MATLAB source code online[58].

Spring design

The main variables for this engineering problem are illustrated in Figure 5 namely, diameter of the wire d, coil diameter D, and number of active coils N. HEO algorithm has been compared with several metaheuristics including CGWO[67], GWO[66], CSA[68], GA3[69], GA4[70], CPSO[71], HPSO[72], G-QPSO[73], PSO[73], DSS-MDE[74], PSO-DE[75], SC[76], UPSO[77,78], ABC[79], TLBO[80], and MBA[81]. The results are obtained from 30 independent runs and 500 iterations. The problem can be declared as:

	PSO		FA		FFPSO		HPSOFF		HFPSO		HEO	
Fn.	Mean	Std.	Mean	Std.	Mean	Std.	Mean	Std.	Mean	Std.	Mean	Std.
10D												
F1	2.4553E + 08	1.3549E + 08	4.3059E + 08	2.8945E + 08	1.6287E + 10	4.9786E + 09	4.8387E + 07	3.4292E + 07	1.3768E + 07	6.6375E + 06	**5.2181E + 05**	**1.9037E + 06**
F2	3.8112E + 04	1.5114E + 04	3.3304E + 04	9.7404E + 03	1.4957E + 08	4.6261E + 08	3.8331E + 04	1.2383E + 04	3.8542E + 04	1.5696E + 04	4.2413E + 05	8.1788E + 05
F3	3.0779E + 02	1.3259E + 00	3.0773E + 02	1.2487E + 00	3.1455E + 02	1.6124E + 00	3.0845E + 02	1.5636E + 00	3.0671E + 02	1.4189E + 00	3.2002E + 02	**4.3765E − 02**
F4	2.2534E + 03	3.5521E + 02	1.5473E + 03	3.2112E + 02	3.1120E + 03	2.6203E + 02	1.7084E + 03	3.0718E + 02	1.3159E + 03	3.9950E + 02	**4.2613E + 02**	**1.3845E + 01**
F5	5.0277E + 02	6.3611E − 01	5.0293E + 02	5.9796E − 01	5.0350E + 02	9.3430E − 01	5.0273E + 02	8.2275E − 01	5.0250E + 02	5.7466E − 01	7.2702E + 02	1.1885E + 01
F6	6.0089E + 02	2.8490E − 01	6.0092E + 02	5.8361E − 01	6.0673E + 02	1.3580E + 00	6.0063E + 02	1.4097E − 01	6.0054E + 02	1.4584E − 01	3.3548E + 03	2.4698E + 03
F7	7.0193E + 02	1.8947E + 00	7.0586E + 02	5.8077E + 00	8.0586E + 02	3.3329E + 01	7.0087E + 02	9.3694E − 01	7.0060E + 02	2.5433E − 01	7.0349E + 02	8.0614E − 01
F8	8.1583E + 02	2.7690E + 01	8.6344E + 02	1.7256E + 02	2.7632E + 05	2.7423E + 05	8.0740E + 02	2.8927E + 00	8.0773E + 02	4.0866E + 00	1.2506E + 04	4.8125E + 04
F9	9.0391E + 02	3.2749E − 01	9.0395E + 02	2.3842E − 01	9.0451E + 02	1.7883E − 01	9.0388E + 02	2.3372E − 01	9.0393E + 02	2.6387E − 01	1.0005E + 03	**5.6373E − 02**
F10	2.9540E + 05	1.9786E + 05	5.3162E + 05	6.5054E + 05	5.1186E + 07	7.7896E + 07	3.5402E + 05	2.9730E + 05	3.3099E + 05	3.3036E + 05	**2.0484E + 03**	**2.7392E + 02**
F11	1.1088E + 03	2.9153E + 00	1.1080E + 03	2.4020E + 00	1.2198E + 03	6.7179E + 01	1.1067E + 03	1.9652E + 00	1.1074E + 03	2.6814E + 00	1.4011E + 03	**3.6109E − 01**
F12	1.4620E + 03	1.1574E + 02	1.3995E + 03	9.1615E + 01	2.1953E + 03	4.3894E + 02	1.4517E + 03	9.5565E + 01	1.3983E + 03	1.0221E + 02	**1.3039E + 03**	**1.4403E − 01**
F13	1.6415E + 03	1.9141E + 01	1.6437E + 03	2.9519E + 01	3.0005E + 03	9.8971E + 02	1.6333E + 03	2.5959E + 01	1.6452E + 03	2.8341E + 01	**1.3427E + 03**	**3.5381E + 00**
F14	1.6076E + 03	4.5254E + 00	1.6111E + 03	3.5980E + 00	1.6770E + 03	4.2292E + 01	1.6053E + 03	5.0554E + 00	1.6021E + 03	5.8221E + 00	4.3467E + 03	1.9972E + 01
F15	1.9149E + 03	1.4570E + 02	1.9269E + 03	7.4514E + 01	2.1840E + 03	1.0567E + 02	1.8365E + 03	1.8650E + 02	1.9233E + 03	1.0398E + 02	**1.6001E + 03**	**2.6114E − 01**

Table 2. Statistical results of HEO algorithm on CEC 2015 for 10D compared to other results from[53].

$$
\begin{aligned}
&\text{Consider } \vec{x} = [\ x_1 \quad x_2 \quad x_3\ ] = [\ d \quad D \quad N\ ],\\
&\text{Minimize } f(\vec{x}) = (x_3 + 2)\, x_2 x_1^2,\\
&\text{Subject to } g_1(\vec{x}) = 1 - \frac{x_2^3 x_3}{717854 x_1^4} \le 0,\\
&g_2(\vec{x}) = \frac{4x_2^2 - x_1 x_2}{12566\,(x_2 x_1^3 - x_1^4)} + \frac{1}{5108 x_1^2} \le 0,\\
&g_4(\vec{x}) = \frac{x_1 + x_2}{1.5} - 1 \le 0.\\
&\text{Variable range } 0.05 \le x_1 \le 2.00,\\
&0.25 \le x_2 \le 1.3,\\
&2.00 \le x_3 \le 15.00
\end{aligned}
\tag{5}
$$

The experimental results are obtained from[67] and used to evaluate the performance of HEO algorithm which is shown in Table 7. As the results show, HEO has converged to the global minimum of this design problem efficiently similar to the other party. According to the results, CGWO has yielded the miumim cost fumction for the spring design problem, however, this result needs more invitigation.

The solution vectors that correspond to the minimum cost function of HEO and CGWO are invistigated to see if they have violations to the constraints that are mentioned in Eq. (5). Table 8 shows that the solution vector of HEO, which corresponds to 0.0126722 cost value, meets the constraints conditions. On the other hand, the solution vector of the CGWO, which corresponds to 0.0119598 cost value, violates the second condition $g_2(\vec{x}) \le 0$. Consequently, the solution of CGWO cannot be the optimum one.

Welded beam design

The Welded beam design is an engineering problem, which is represented in Figure 6, where it has four variables and the objective is to reduce the fabrication cost. The problem can be expressed as follows:

$$
\begin{aligned}
&\text{Consider } \vec{x} = [\ x_1 \quad x_2 \quad x_3 \quad x_4\ ] = [\ h \quad l \quad t \quad b\ ],\\
&\text{Minimize } f(\vec{x}) = 1.10471 x_1^2 x_2 + 0.04811 x_3 x_4 (14.0 + x_2),\\
&\text{subject to } g_1(\vec{x}) = \tau(\vec{x}) - \tau_{\max} \le 0,\\
&g_2(\vec{x}) = \sigma(\vec{x}) - \sigma_{\max} \le 0,\\
&g_3(\vec{x}) = \delta(\vec{x}) - \delta_{\max} \le 0,\\
&g_4(\vec{x}) = x_1 - x_4 \le 0,\\
&g_5(\vec{x}) = P - P_C(\vec{x}) \le 0,\\
&g_6(\vec{x}) = 0.125 - x_1 \le 0,\\
&g_7(\vec{x}) = 0.10471 x_1^2 + 0.04811 x_3 x_4 (14.0 + x_2) - 0.5,
\end{aligned}
\tag{6}
$$

where $\tau(\vec{x}) \sqrt{\tau'^2 + 2\tau'\tau'' \frac{x_2}{2R} + \tau''^2}$, $\tau' = \frac{P}{\sqrt{2} x_1 x_2}$, $\tau'' = \frac{MR}{J}$, $M = P\left(L + \frac{x_2}{2}\right)$, $R = \sqrt{\frac{x_2^2}{4} + \left(\frac{x_1 + x_3}{2}\right)^2}$, $J = 2\left\{\sqrt{2} x_1 x_2 \left[\frac{x_2^2}{4} + \left(\frac{x_1 + x_3}{2}\right)^2\right]\right\}$, $\sigma(\vec{x}) = \frac{6PL}{x_4 x_3^2}$, $\delta(\vec{x}) = \frac{6PL^3}{E x_3^2 x_4}$, $P_C(\vec{x}) = \frac{4.013 \sqrt{\frac{x_3^2 x_4^6}{36}}}{L^2} \left(1 - \frac{x_3}{2L}\sqrt{\frac{E}{4G}}\right)$, $P = 6000lb, L = 14in.,\ \delta_{\max} = 0.25in., E = 30 * 10^6 psi., G = 12 * 10^6 psi., \tau_{\max} = 13600psi., \sigma_{\max} = 30000psi.,$ $0.1 \le x_1 \le 2, 0.1 \le x_2 \le 10, 0.1 \le x_3 \le 10, 0.1 \le x_4 \le 2$.

Table 9 shows the comparison results among HEO and other algorithms from the litereture[66]. The adopted algorithms for this study are gray wolf optimizer GWO, gravitational search algorithm GSA[82], genetic algorithm GA[83–85], harmony search HS[86], random (Richardson's random method), Simplex method, Davidon-Fletcher-Powell, APPROX[87]. For 30 independent runs and 500 iterations, the results reveal that HEO estimates a design with the minimum cost compared to other algorithms. In addition, the performance of HEO on this problem has been evaluated using another study[67] and similarly, it has yielded the minimum cost, as shown in Table 10.

However, results shown in Tables 9 and 10 should be discussed in term of the validity and optimality of the solutions. Table 11 shows the number of violations to the costraints mentioned in Eq. (6), it reveals that all the best solution of the algorithms have violated some conditions that the value of the constraits should be less than or equal zero. We can say there is no optimal solution for this problem yet and although CGWO has the minimum cost function, it actually has the worst solution among the party beacuae it violated the constraints 4 times out of 7 conditions.

Although the HEO algorithm yielded a low-cost value of 1.40333 in the welded beam design problem, a detailed feasibility check reveals that this solution, like many others reported in the literature, violates three of the seven design constraints. This indicates that despite the numerical optimality in terms of the objective function, the solution does not fully satisfy all feasibility conditions. Table 11 explicitly lists the violated constraints for all methods. Notably, CGWO, which produced an even lower cost of 1.72545, violates four constraints and is therefore less reliable despite its apparent optimality. These results reinforce the conclusion that none of the compared algorithms, including HEO, achieved a truly feasible global optimum. Hence, future studies should

Fn.	PSO		FA		FFPSO		HPSOFF		HFPSO		HEO	
	Mean	Std.	Mean	Std.	Mean	Std.	Mean	Std.	Mean	Std.	Mean	Std.
30D												
F1	3.9049E + 09	1.6017E + 09	2.8899E + 10	5.8485E + 09	9.3283E + 10	1.2799E + 10	4.7539E + 09	1.3400E + 09	1.1795E + 09	6.2432E + 08	**3.8729E + 06**	**1.2163E + 06**
F2	9.9760E + 04	2.5812E + 04	1.3418E + 05	2.6344E + 04	6.9430E + 06	1.4507E + 07	9.7376E + 04	2.0814E + 04	8.5653E + 04	2.0924E + 04	1.3310E + 06	2.7180E + 06
F3	3.3113E + 02	2.8347E + 00	3.3850E + 02	1.9604E + 00	3.4771E + 02	1.9232E + 00	3.3059E + 02	2.6174E + 00	3.2638E + 02	4.1408E + 00	**3.2020E + 02**	**1.8740E − 01**
F4	7.7928E + 03	5.2384E + 02	8.0330E + 03	4.2608E + 02	9.6696E + 03	4.0262E + 02	6.8199E + 03	8.5746E + 02	5.1202E + 03	6.7938E + 02	**5.1944E + 02**	**4.5980E + 00**
F5	5.0418E + 02	7.2160E − 01	5.0425E + 02	5.1916E − 01	5.0586E + 02	1.1990E + 00	5.0422E + 02	6.1627E − 01	5.0410E + 02	8.1315E − 01	2.8129E + 03	2.2803E + 02
F6	6.0096E + 02	1.8792E − 01	6.0410E + 02	2.2901E − 01	6.0755E + 02	6.6978E − 01	6.0090E + 02	1.3068E − 01	6.0076E + 02	9.5338E − 02	2.8345E + 05	4.8039E + 05
F7	7.0405E + 02	2.0114E + 00	7.6997E + 02	1.2225E + 01	8.9401E + 02	2.6905E + 01	7.0750E + 02	4.2654E + 00	**7.0074E + 02**	3.3250E − 01	7.1285E + 02	**3.1369E − 01**
F8	4.0013E + 03	**3.3477E + 03**	2.3491E + 06	1.4471E + 06	1.6032E + 08	1.1620E + 08	1.7191E + 04	3.3261E + 04	**2.6354E + 03**	4.4907E + 03	9.9184E + 04	1.1744E + 04
F9	9.1358E + 02	2.3378E − 01	9.1381E + 02	2.8076E − 01	9.1427E + 02	**1.8457E − 01**	9.1365E + 02	3.1664E − 01	**9.1337E + 02**	2.2728E − 01	1.0036E + 03	6.9823E − 01
F10	7.5602E + 06	3.1209E + 06	2.9938E + 07	1.4513E + 07	3.9391E + 08	1.6830E + 08	1.1337E + 07	6.3462E + 06	5.4690E + 06	3.3465E + 06	**4.4381E + 05**	**2.4973E + 05**
F11	1.1614E + 03	4.1043E + 01	1.2889E + 03	3.7640E + 01	2.1094E + 03	4.6591E + 02	1.1551E + 03	2.8498E + 01	**1.1336E + 03**	2.2415E + 01	1.4182E + 03	**5.7347E + 00**
F12	2.2417E + 03	1.8647E + 02	2.9655E + 03	2.7960E + 02	4.9876E + 05	6.1606E + 05	2.0617E + 03	2.0746E + 02	1.7752E + 03	1.4548E + 02	**1.3090E + 03**	**6.0340E − 01**
F13	1.7719E + 03	2.6527E + 01	1.9596E + 03	8.1067E + 01	3.6329E + 03	7.7365E + 02	1.7390E + 03	2.3388E + 01	1.6866E + 03	1.6524E + 01	**1.4117E + 03**	**2.6298E + 00**
F14	1.6644E + 03	1.8615E + 01	1.7522E + 03	2.6072E + 01	2.1683E + 03	1.4580E + 02	1.6711E + 03	2.8498E + 01	**1.6469E + 03**	**9.7379E + 00**	3.4032E + 04	1.0034E + 02
F15	2.5522E + 03	1.7142E + 02	2.8256E + 03	5.5492E + 01	3.9013E + 03	4.4677E + 02	2.6529E + 03	1.6852E + 02	2.4467E + 03	2.1048E + 02	**1.6002E + 03**	**4.8410E − 01**

Table 3. Statistical results of HEO algorithm on CEC 2015 for 30D compared to other results from[53].

No	Description	Function	d	Range	*fmin*
F16	Unimodal	$f = \sum_{i=1}^{n} ([x_i + 0.5])^2$	30	[−100, 100]	0
F17		$f = \sum_{i=1}^{n} ix_i^4 + random[0, 1)$	30	[−1.28, 1.28]	0
F18	Multimodal	$f = \sum_{i=1}^{n} -x_i \sin\left(\sqrt{\lvert x_i \rvert}\right)$	30	[−500, 500]	-418.9829×5
F19		$f = \sum_{i=1}^{n} \left[x_i^2 - 10\cos(2\pi x_i) + 10\right]$	30	[−5.12, 5.12]	0
F20		$f = \left(\frac{1}{500} + \sum_{j=1}^{25} \frac{1}{j + \sum_{i-1}^{2} (x_i - a_{ij})^2}\right)^{-1}$	2	[−65, 65]	1
F21		$f = \left(x_2 - \frac{5.1}{4\pi^2} x_1^2 + \frac{5}{\pi} x_1 - 6\right)^2 + 10\left(1 - \frac{1}{8\pi}\right)\cos x_1 + 10$	2	[−5, 5]	0.398
F22		$f = -\sum_{i=1}^{4} C_i \exp\left(-\sum_{j=1}^{6} a_{ij} (x_j - p_{ij})^2\right)$	6	[0, 1]	3.32
F23		$f = -\sum_{i=1}^{10} \left[(x - a_i)(x - a_i)^T + c_i\right]^{-1}$	4	[0, 10]	10.5363

Table 4. Unimodal and multimodal optimization problems[66].

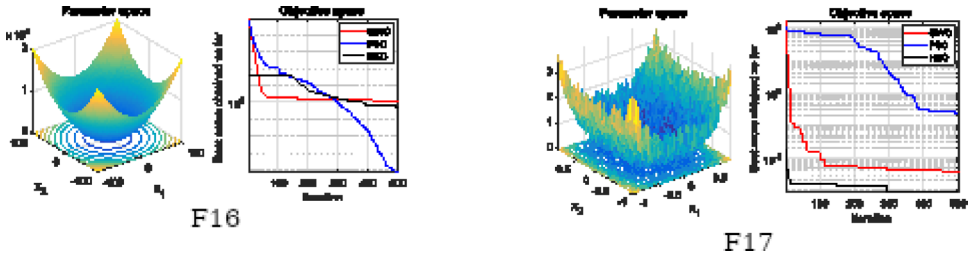


Fig. 3. Unimodal test functions.

focus on more effective constraint-handling techniques to ensure feasibility in highly constrained engineering problems like the welded beam design.

Pressure vessel design

Pressure vessel design is a common optimization problem whose objective is to minimize the manufacturing cost of the vessel. The cost including mass, welding, and forming, and there are range of solution flactuates between 6000~8000. This design problem is shown in Figure 7 and formulated as.

$$
\begin{aligned}
&\text{Consider } \vec{x} = [\ x_1 \quad x_2 \quad x_3 \quad x_4\ ] = [\ T_s \quad T_h \quad R \quad L\ ],\\
&\text{Minimise } f(\vec{x}) = 0.6224x_1x_2x_3 + 1.7781x_2x_3^2 + 3.1661x_1^2x_4 + 19.84x_1^2x_3,\\
&\text{Subject to } g_1(\vec{x}) = -x_1 + 0.0193x_3 \le 0,\\
&g_2(\vec{x}) = -x_3 + 0.00954x_3 \le 0,\\
&g_3(\vec{x}) = -\pi x_3^2 x_4 - \frac{4}{3}\pi x_3^3 + 1296000 \le 0,\\
&g_4(\vec{x}) = x_4 - 240 \le 0,\\
&\text{Variable range } 0.1 \le x_1 \le 99,\\
&0.1 \le x_2 \le 99,\\
&10 \le x_3 \le 200,\\
&10 \le x_4 \le 200,
\end{aligned}
\tag{7}
$$

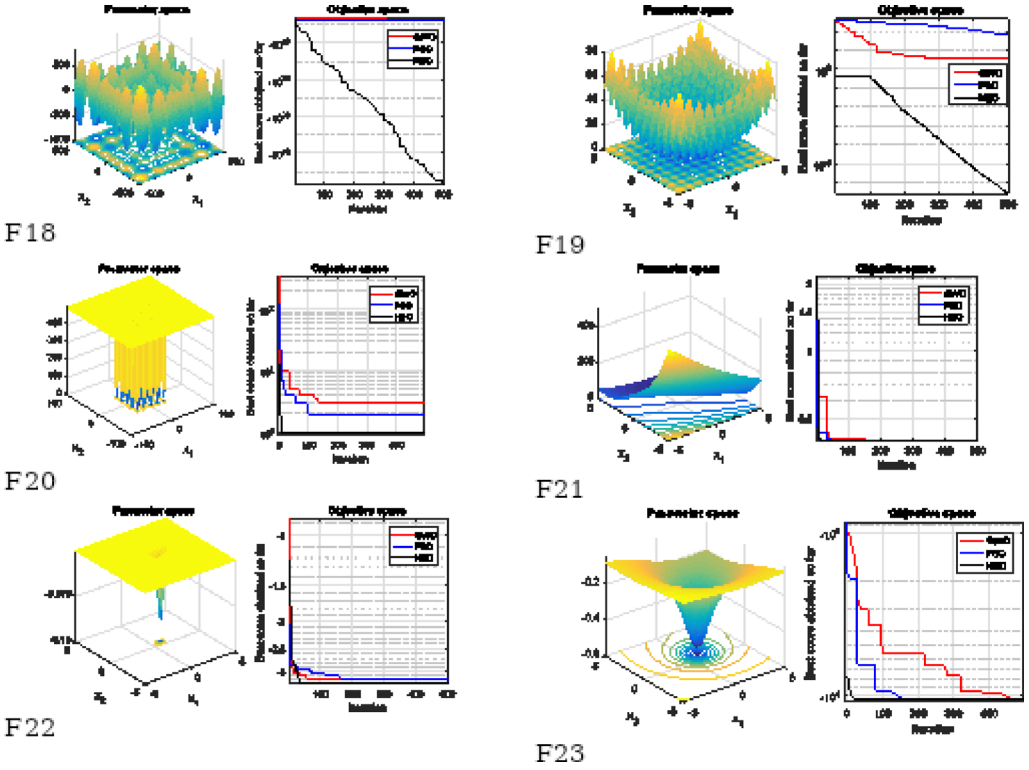


Fig. 4. Multimodal test function.

Function	Name	Expression	d	Range	Fmin
F24	Bohachevsky	$f(x) = x_1^2 + 2x_2^2 - 0.3\cos(3\pi.x_1) - 0.4\cos(4\pi.x_2) + 0.7$	2	[−100, 100]	0
F25	Booth	$f(x) = (x_1 + 2x_2 - 7)^2 + (2x_1 + x_2 - 5)^2$	2	[−10, 10]	0
F26	Matyas	$f(x) = 0.26\left(x_1^2 + x_2^2\right) - 0.48x_1.x_2$	2	[−10, 10]	0
F27	Easom	$f(x) = -\cos(x_1).\cos(x_2).\exp\left(-(x_1 - \pi)^2 - (x_2 - \pi)\right)$	2	[−100, 100]	−1
F28	Beale	$f(x) = (1.5 - x_1 + x_1x_2)^2 + \left(2.25 - x_1 + x_1x_2^2\right)^2 + \left(2.625 - x_1 + x_1x_2^3\right)^2$	2	[−4.5, 4.5]	0
F29	Schaffer N.2	$f(x) = 0.5 + \frac{\sin^2\left(\sqrt{x_1^2+x_2^2}\right)-0.5}{\left(1+0.001\left(x_1^2+x_2^2\right)\right)^2}$	2	[−100, 100]	0
F30	Six-hump Camel	$f(x) = 4x_1^2 - 2.1x_1^4 + \frac{1}{3}x_1^6 + x_1x_2 + \left(-4 + 4x_2^2\right)x_2^2$	2	[−2, 2]	−1.03163

Table 5. Small-scale benchmarks[59].

The statistical results are taken from the litreture[66] where the adopted methds include PSO[71], GA[70,88,89], ES[90], DE[91], ACO[92] made for 30 independent runs and 500 iterations. In addition, mathematical methods augmented lagrangian multiplier[93] and branch-and-bound[94] have been included. HEO algorithm has been tested using these avialable data whose results are shown in Table 12. According to the results, HEO has the best ability to find a design with the minimum cost. Besides, the performance of HEO has been compared with another study given in Table 13[67] where the proposed algorithm is also well-performed comparing with other methds.

Table 14 illustrates the number of violations to the constraints in Eq. (7). It is clear that CGWO has returned solution which violates the conditions 2 times out of 4 conditins. Thus, the solution of CGWO can not be the obtimal one. On the other hand, HEO has returned the minimum cost function among the party and its best solution did not violate any constraints. Thus, we could consider that the solution of the proposed HEO algorithm is the most reliable solution for the pressure vessel design problem.

Function		PSO	BA	FF	MOV	KH	SSA	HEO
F24	Best	4.4298E − 14	1.6438E+00	4.9443E − 08	1.0021E − 05	2.8890E − 08	**0**	**0**
	Worst	1.3643E − 09	4.3439E+02	9.1103E − 06	1.5693E − 03	1.4369E − 06	**0**	**0**
	Mean	1.4898E − 10	6.9719E+01	3.4682E − 06	3.5952E − 04	2.2989E − 07	**0**	**0**
	STD	2.7559E − 10	1.1194E+02	2.6636E − 06	3.7081E − 04	2.9457E − 07	**0**	**0**
F25	Best	1.3482E − 14	3.0619E − 11	2.7382E − 10	4.2336E − 09	5.9289E − 12	**1.2622E − 29**	6.2999E − 12
	Worst	1.2419E − 09	2.0412E − 09	1.9998E − 08	2.9865E − 06	1.5568E − 09	**8.1255E − 24**	2.9285E − 09
	Mean	8.7865E − 11	6.7996E − 10	5.4966E − 09	5.8026E − 07	1.9913E − 10	**9.5859E − 25**	1.1049E − 09
	STD	2.3629E − 10	5.4689E − 10	4.8029E − 09	6.5125E − 07	3.1189E − 10	**1.7996E − 24**	7.9369E − 10
F26	Best	8.6209E − 17	1.4036E − 12	2.9676E − 12	2.0125E − 10	2.1689E − 14	1.5111E − 29	2.7536E − 24
	Worst	7.1849E − 12	2.2319E − 10	1.2805E − 09	4.0157E − 08	6.8392E − 11	2.0707E − 24	1.8743E − 14
	Mean	8.4738E − 13	2.9659E − 11	3.1396E − 10	1.3148E − 08	8.8694E − 12	1.542E − 25	1.2314E − 15
	STD	1.5676E − 12	4.8089E − 11	2.9301E − 10	1.1549E − 08	1.2854E − 11	4.7571E − 25	3.6743E − 15
F27	Best	**− 1**	**− 1**	**− 1**	**− 1**	**− 1**	**− 1**	**− 1**
	Worst	**− 1**	0	0	0	0	**− 1**	**− 1**
	Mean	**− 1**	− 3.3347E − 02	− 7.3333E − 01	− 9.6664E − 01	− 9.6666E − 01	**− 1**	**− 1**
	STD	2.8316E − 11	1.8257E − 01	4.4978E − 01	1.8257E − 01	1.8257E − 01	**0**	**0**
F28	Best	1.3624E − 14	2.8649E − 11	1.2360E − 11	9.2179E − 09	4.4607E − 13	**2.1832E − 29**	4.8073E − 13
	Worst	4.9376E − 01	7.6207E − 01	3.9985E − 09	7.6207E − 01	1.5103E − 09	**2.7633E − 20**	0.7620
	Mean	6.2888E − 02	1.5241E − 01	8.4259E − 10	5.0805E − 02	1.7322E − 10	**9.5584E − 22**	0.2929
	STD	1.6315E − 01	3.1004E − 01	7.7095E − 10	1.9334E − 01	3.0481E − 10	**5.0400E − 21**	0.3678
F29	Best	**0**	9.7159E − 03	4.6615E − 03	2.1517E − 06	1.2317E − 07	**0**	**0**
	Worst	9.7159E − 03	3.7329E − 01	3.8682E − 02	9.7159E − 03	**2.0569E − 05**	9.7159E − 03	8.3667E − 05
	Mean	3.5625E − 03	1.4118E − 01	1.1809E − 02	1.3113E − 03	**3.3604E − 06**	9.7159E − 04	**2.9961E − 06**
	STD	4.7621E − 03	1.0699E − 01	7.3682E − 03	3.3529E − 03	**4.1334E − 06**	2.9646E − 03	1.5278E − 05
F30	Best	**− 1.03163**	**− 1.03163**	**− 1.03163**	**− 1.03163**	**− 1.03163**	**− 1.03163**	− 1.031628
	Worst	**− 1.03163**	**− 0.21546**	**− 1.03163**	**− 1.03163**	**− 1.03163**	**− 1.03163**	− 1.031628
	Mean	**− 1.03163**	**− 0.95001**	**− 1.03163**	**− 1.03163**	**− 1.03163**	**− 1.03163**	− 1.031628
	STD	8.2049E − 11	2.4904E − 01	3.9448E − 09	1.6383E − 07	6.3582E − 10	**4.5168E − 16**	1.43835E − 10

Table 6. Details of statistical performance of HEO and other metaheuristics[60] in small-scale optimization problems.

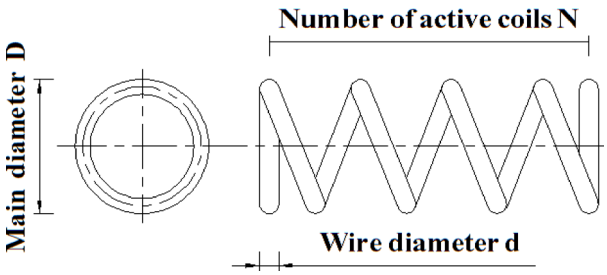


Fig. 5. Closed coil helical spring design parameters.

10-bar space truss

A common benchmark in structural optimization is the 10-bar cantilever truss shown in Figure 8 shows the 10-bar space truss design problem[95].

In this problem, the cross-sectional areas are permitted to change between 0.1 and 35.0 in^2 in the continuous variable case. The 10-bar truss design problem is a structural optimization benchmark that seeks to minimize the total weight of a truss structure under stress and displacement constraints. Table 15 compares the performance of the HEO algorithm with several well-established metaheuristics, including Genetic Algorithm (GA), Particle Swarm Optimization (PSO), and Teaching-Learning-Based Optimization (TLBO). The key metrics analyzed are the best-obtained weight (lb), the average weight (W_{avg}), and the standard deviation (W_{Stdv}) over multiple runs.

Algorithm	Worst	Mean	Best	STD
HEO	0.0164353	0.0141220	0.0126722	0.0012482
CGWO	0.0121791	0.0121749	0.0119598	1.039E−05
GWO	0.0122515	0.0121836	0.0126660	1.085E−05
CSA	0.0126701	0.0127690	0.0126650	1.357E−06
GA3	0.0128220	0.0127690	0.0127048	3.940E−05
GA4	0.0129730	0.0127420	0.0126810	5.90E−05
CPSO	0.0129240	0.0127330	0.0126747	5.20E−04
HPSO	0.0127190	0.0127072	0.0126652	1.58E−05
G-QPSO	0.0177590	0.0135240	0.0126650	0.001268
QPSO	0.0181270	0.0138540	0.0126690	0.001341
PSO	0.0718020	0.0195550	0.0128570	0.011662
DSS-MDE	0.0127382	0.0126693	0.0126652	1.25E−05
PSO-DE	0.0126653	0.0126652	0.0126652	1.2E−08
SC	0.0167172	0.0129226	0.0126692	5.9E−04
UPSO	N.A.	0.0229400	0.0131200	7.2E−03
$(\mu + \lambda) - ES$	N.A.	0.0131650	0.0126890	3.9E−04
ABC	N.A.	0.0127090	0.0126650	0.01281
TLBO	N.A.	0.0126657	0.0126650	N.A.
MBA	0.0129000	0.0127130	0.0126650	6.3E−05

Table 7. Comparing HEO with CGWO and other algorithms[66]. *N.A* not available.

Algorithm	$(\vec{x}_*)^T$	$g_1(\vec{x})$	$g_2(\vec{x})$	$g_3(\vec{x})$	$g_4(\vec{x})$	No. of violations
HEO	0.052149	−0.0001	−0.0001	−4.0742	−0.7200	0
	0.367836					
	10.66809					
CGWO	0.052796	−0.8663	0.9018	−4.7302	−0.4285	1
	0.804380					
	2.000000					

Table 8. Violations of the best Solutin vectors of HEO and CGWO algorithms to the constraints.

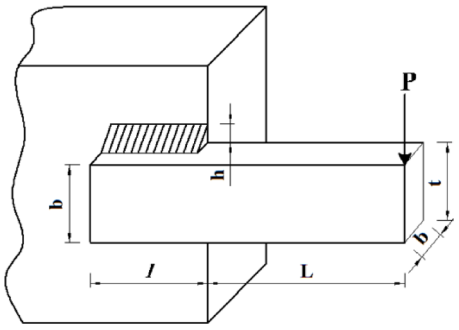


Fig. 6. Welded beam design problem.

As the results show, HEO has achieved a best weight of 5,060.77 lb, which is very close to the optimal solutions produced by HPSO (5,060.92 lb) and ABC-AP (5,060.88 lb). This suggests that HEO is highly competitive in finding near-optimal solutions for structural optimization tasks. However, the best weight obtained by PSO (5,059.85 lb) and HS (5,057.88 lb) is slightly better than that of HEO. The differences are small, indicating that all algorithms performed similarly, with HEO closely trailing the top performers. The results show that HEO performed competitively against several well-known metaheuristics on the 10-bar truss design problem.

Algorithm	Optimum variables				Optimum cost
	h	l	t	b	
HEO	**2**	**9.985541**	**9.208974**	**2**	**1.40333**
GWO	0.205676	3.478377	9.03681	0.205778	1.72624
GSA	0.182129	3.856979	10.00000	0.202376	1.879952
GA[83]	N/A	N/A	N/A	N/A	1.8245
GA[84]	N/A	N/A	N/A	N/A	2.3800
GA[85]	0.2489	6.1730	8.1789	0.2533	2.4331
HS	0.2442	6.2231	8.2915	0.2443	2.3807
Random	0.4575	4.7313	5.0853	0.6600	4.1185
Simplex	0.2792	5.6256	7.7512	0.2796	2.5307
David	0.2434	6.2552	8.2915	0.2444	2.3841
APPROX	0.2444	6.2189	8.2915	0.2444	2.3815
CGWO[67]	0.343891	1.883570	9.03133	0.212121	1.72545

Table 9. Comparing HEO optimum solution with other methods mentioned in[66] on the welded beam design problem. Significant values are in bold. *N.A.* not available.

Algorithm	Worst	Mean	Best	STD
HEO	74.92160	26.81078	**1.403336**	18.0581
CGWO	2.435700	2.428900	1.725450	1.35780
CPSO	**1.782143**	**1.748831**	1.728024	**1.29E − 2**
GA4	1.993408	1.792654	1.728226	7.47E − 2
GA3	1.785835	1.771973	1.748309	1.12E − 2
CDE	N.A.	1.768150	1.733460	N.A.
UPSO	N.A.	2.837210	1.921990	0.68300
SC	6.399678	3.002588	2.385434	9.60E − 1

Table 10. Comparing the performance of HEO with other algorithms adopted by[67]. Significant values are in bold.

Algorithm	$g_1(\vec{x})$	$g_2(\vec{x})$	$g_3(\vec{x})$	$g_4(\vec{x})$	$g_5(\vec{x})$	$g_6(\vec{x})$	$g_7(\vec{x})$	No. of violations
HEO	− 11,185	2.517E + 07	164.39	− 1.5446	6000	− 0.33041	− 0.34261	3
GWO	11,561	− 8.2856	− 0.054054	− 0.000102	6000	− 0.080676	1.5654	3
GSA	6608.5	− 5095.9	− 0.087293	− 0.020247	6000	− 0.057129	1.8402	3
GA[85]]	− 9954.1	− 255.58	− 0.05567	− 0.0044	6000	− 0.1239	1.7171	2
HS	− 9933.1	8.2631	− 0.053946	− 0.0001	6000	− 0.1192	1.6786	3
Random	− 10,407	− 470.68	− 0.057075	− 0.2025	6000	− 0.3325	2.6036	2
Simplex	− 9739.3	2.3645	− 0.053985	− 0.0004	6000	− 0.1542	1.7761	3
David	− 9954.8	− 4.0152	− 0.054026	− 0.001	6000	− 0.1184	1.6794	2
APPROX	− 9931.7	− 4.0152	− 0.054026	0	6000	− 0.1194	1.6795	2
CGWO	2.0045E + 05	− 869.8	− 0.059683	0.13177	6000	− 0.21889	1.6351	4

Table 11. Violations to constraints of the best solution vectors of the algorithm in Tables 9 and CGWO in Table 10.

It achieved a near-optimal solution comparable to those of HPSO and ABC-AP while demonstrating better consistency than standard PSO. However, HEO's slightly higher average weight and standard deviation indicate that further refinement of its search mechanism may be needed to reduce variability and improve the average-case performance. Overall, HEO is a strong contender for structural optimization tasks, providing robust solutions with competitive weights. While it does not outperform all algorithms in every metric, its results are very close to the best-known solutions, making it a reliable option for truss design and similar engineering applications.

HEO algorithm on CEC2020

This section evaluates the performance of the proposed HEO algorithm on a subset of constrained optimization problems from the CEC2020 testbed. These real-world problems are characterized by varying numbers

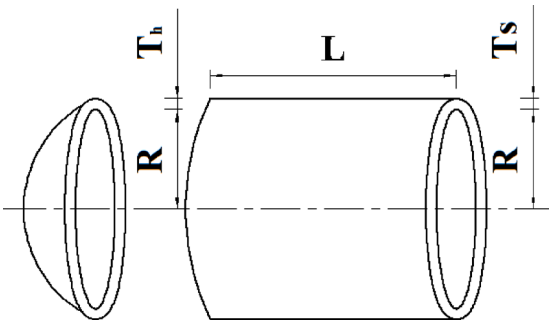


Fig. 7. Pressure vessel design parameters.

Algorithm	Optimum variable				Optimum cost
	Ts	Th	R	L	
HEO	**0.778765**	**0.385292**	**40.349152**	**199.767993**	**5891.3852**
GWO	0.812500	0.434500	42.089181	176.758731	6051.5639
GSA	1.125000	0.625000	55.9886598	84.4542025	8538.8359
PSO[71]	0.812500	0.437500	42.091266	176.746500	6061.0777
GA[70]	0.812500	0.434500	40.323900	200.000000	6288.7445
GA[88]	0.812500	0.437500	42.097398	176.654050	6059.9463
GA[89]	0.937500	0.500000	48.329000	112.679000	6410.3811
ES[90]	0.812500	0.437500	42.098087	176.640518	6059.7456
DE[91]	0.812500	0.437500	42.098411	176.637690	6059.7340
ACO[92]	0.812500	0.437500	42.103624	176.572656	6059.0888
Lagrangian Multiplier[93]	1.125000	0.625000	58.291000	43.6900000	7198.0428
Branch-bound[94]	1.125000	0.625000	47.700000	117.701000	8129.1036

Table 12. Best solution of HEO for pressure vessel problem compared to other methods[66]. Significant values are in bold.

of equality constraints (h), inequality constraints (g), and dimensionality (D). Table 16 summarizes these problems, which include complex engineering applications such as heat exchanger network design, reactor network optimization, and truss structure design. In Table 17, the performance of HEO is compared with other state-of-the-art algorithms, including IUDE, MAgES, and iLSHADE, across several benchmarks. The comparison considers both the best objective function values (Best) and standard deviations (Std) over multiple runs to assess the robustness and efficiency of the algorithms.

For AF1, HEO has achieved the optimal objective value of 1.89E+02, matching IUDE and MAgES. However, the standard deviation (2.47E+02) indicates variability in HEO's performance across runs, suggesting potential issues with premature convergence. On AF2, HEO has performed slightly worse than its competitors, yielding 7.91E+03, while the best result across other algorithms was 7.05E+03. HEO found a near-optimal solution of -3.90E-01, better than most competitors except for MAgES, which closely matched the global minimum. The variability of HEO (Std = 6.60E-03) was lower than that of MAgES, indicating consistent performance. On problems like AF5, HEO has returned the optimal solution 2.00E+00 with negligible variability, matching all other algorithms. Similarly, for AF6 and AF7, HEO has consistently found optimal solutions with small deviations, demonstrating its reliability on well-behaved problem landscapes. In higher-dimensional applications, such as AF10 (Multi-product Batch Plant) and AF8 (Two-reactor Problem), HEO's performance lagged slightly. While it has produced competitive results (e.g., 5.85E+04 for AF10), its standard deviation was higher than that of the best-performing algorithm, iLSHADE. For structural design problems like AF11 and AF13, HEO has delivered results that were the best-known solutions from other algorithms, with low variability across runs. These results highlight the algorithm's robustness for constrained mechanical engineering applications. The results from Table 17 demonstrate that HEO is competitive with leading algorithms across many of the benchmark problems. In fact, it has successfully achieved optimal or near-optimal solutions for problems like AF1, AF5, and AF11. However, the algorithm has shown some variability in performance for complex, high-dimensional problems (e.g., AF2 and AF10). The higher standard deviation observed in some cases suggests that HEO might benefit from improved parameter tuning or hybridization with other techniques to enhance its stability. Overall, HEO proves to be a promising algorithm, especially for constrained engineering design problems. Its ability to match or closely approach the performance of state-of-the-art algorithms like iLSHADE highlights its potential, but further refinement may be necessary to improve consistency on more challenging, high-dimensional tasks.

Algorithm	Worst	Mean	Best	Std
HEO	7304.6933	6736.7996	**5891.3852**	464.3687
CGWO	6188.110	5783.582	5034.180	254.505
GWO	6395.360	6159.320	6051.563	379.674
$(\mu + \lambda) - ES$	N.A.	6379.938	6059.701	210.000
CSA	7332.841	6342.499	6059.714	384.945
HPSO	6288.677	6099.932	6059.714	86.200
PSO-DE	N.A.	6059.714	6059.714	N.A.
ABC	N.A.	6245.308	6059.714	205.000
TLBO	N.A.	6059.714	6059.714	N.A.
G-QPSO	7544.492	6440.378	6059.720	448.471
QPSO	8017.281	6440.378	6059.720	479.267
CDE	6371.045	6085.230	6059.734	43.0130
GA4	6469.322	6177.253	6059.946	130.929
CPSO	6363.804	6147.133	6061.077	86.4500
UPSO	9387.770	8016.370	6154.700	745.869
GA3	6308.497	6293.843	6288.744	7.41330
PSO	14076.324	8756.680	6693.721	1492.56

Table 13. Performance of HEO for pressure vessel problem compared to other methods[67]. Significant values are in bold.

Algorithm	$(\vec{x}_*)^T$	$g_1(\vec{x})$	$g_2(\vec{x})$	$g_3(\vec{x})$	$g_4(\vec{x})$	No. of violations
HEO	0.778765	0.0	0.0	−914.0754	−40.2320	0
	0.385292					
	40.349152					
	199.767993					
GWO	0.812500	−0.00017	−0.03296	−40.61682	−63.24126	0
	0.434500					
	42.089181					
	176.758731					
CGWO	1.187150	0.15820475	0.0650095500	−2.4186	−0.0023	2
	0.600000					
	69.707500					
	7.7984400					

Table 14. Violations of HEO, GWO, and CGWO to constraints.

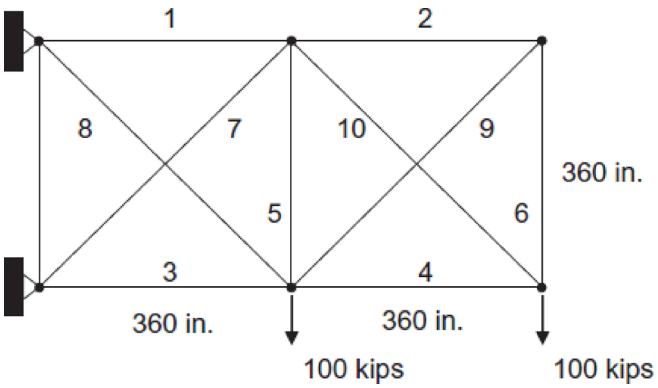


Fig. 8. Configuration of 10-bar space truss.

Variables	GA[96]	PSO[97]	HPSO[98]	HS[99]	TLBO[95]	SAHS[100]	ABC-AP[101]	HEO
Weight (lb)	5,076.31	5,059.85	5,060.92	5,057.88	5,060.973	5,061.42	5,060.88	5,060.77
W_{avg} (lb)	–	5067.51	–	–	5064.808	5061.95	–	5,068.40
W_{Stdv} (lb)	–	17.509	–	–	6.3707	0.71	–	7.86

Table 15. Performance of HEO on 10-bar space truss problem.

Prob	Application	h	g	D	$f(x^*)$
AF1	Heat Exchanger Network Design (case 1)	8	0	9	1.8931162966E + 02
AF2	Heat Exchanger Network Design (case 2)	9	0	11	7.0490369540E + 03
AF3	Reactor Network Design	4	1	6	− 3.8826043623E-01
AF4	Blending-Pooling-Separation problem	32	0	38	1.8638304088E + 00
AF5	Process synthesis problem	0	2	2	2.0000000000E + 00
AF6	Process synthesis and design problem	1	1	3	2.5576545740E + 00
AF7	Process flow sheeting problem	0	3	3	1.0765430833E + 00
AF8	Two-reactor Problem	4	4	7	9.9238463653E + 01
AF9	Process design Problem	0	3	5	2.6887000000E + 04
AF10	Multi-product batch plant	0	10	10	5.3638942722E + 04
AF11	Weight Minimization of a Speed Reducer	0	11	7	2.9944244658E + 03
AF12	Three-bar truss design problem	0	3	2	2.6389584338E + 02
AF13	Multiple disk clutch brake design problem	0	7	5	2.3524245790E − 01

Table 16. Constrained optimization problems from CEC2020 testbed.

Exploration and exploitation

The success of any optimization algorithm depends on its ability to balance exploration (searching new areas of the solution space) and exploitation (refining the search within promising regions). In the context of the HEO algorithm, these two phases are intricately modeled to mimic the behavior of hares evading predators, leveraging both randomness and strategic movements. Exploration in HEO is achieved through the integration of Levy flight dynamics, a well-known stochastic process characterized by long jumps interspersed with short steps. This mechanism allows HEO to:

1. Escape local optima: By making long-distance jumps, the algorithm can avoid stagnation in suboptimal regions.
2. Enhance global search: The random nature of Levy flights ensures that diverse areas of the search space are sampled, increasing the likelihood of locating global optima.
3. Widen the search coverage: Levy flights create non-linear trajectories, providing a probabilistically efficient method for exploring vast and complex landscapes.

Mathematically, Levy flights are incorporated into the hare's movement, guiding the solution update process by generating random steps influenced by the Levy distribution. Exploitation in HEO focuses on intensifying the search around promising solutions. This is achieved through:

1. Random Directional Changes: Modeled after the hare's sudden directional shifts, this mechanism fine-tunes the search by concentrating efforts in promising regions while still introducing localized randomness to avoid premature convergence.
2. Distance Reduction Strategy: As iterations progress, the distance between the hare (solution) and the wolf (optimum approximation) is gradually reduced, ensuring a more refined search.)
3. Population Sorting and Selection: At each iteration, solutions are sorted based on their fitness values. The best solutions are retained, and the worst are replaced or modified, ensuring that the algorithm focuses on the most promising areas of the search space.

The HEO algorithm maintains a dynamic balance between exploration and exploitation by:

1. Dynamic Control of Search Intensity: The step size and directionality are influenced by the current iteration and solution diversity. Early iterations prioritize exploration, while later iterations emphasize exploitation.
2. Hybrid Strategy: The combination of Levy flights and directional shifts ensures that the algorithm neither over-explores nor converges prematurely, achieving a balance which is crucial for complex optimization problems.

This balance enables HEO to:

Application	Algorithm							
	HEO		IUDE		εMAgES		iLSHADEε	
	Best	Std	Best	Std	Best	Std	Best	Std
AF1	1.89E + 02	2.47E + 02	1.89E + 02	8.06E + 01	1.89E + 02	2.23E + 02	1.90E + 02	1.93E + 01
AF2	7.91E + 03	7.23E + 03	7.05E + 03	7.54E + 02	7.05E + 03	7.50E + 02	7.05E + 03	2.13E-11
AF3	-3.90E-01	6.60E-03	− 2.86E-01	1.82E-01	− 3.88E-01	2.86E-01	− 3.75E-01	4.61E-04
AF4	1.51E + 00	3.03E-01	1.71E + 00	1.98E-01	2.09E + 00	3.39E-01	1.72E + 00	1.63E-01
AF5	2.00E + 00	1.58E-09	2.00E + 00	6.41E-17	2.00E + 00	1.52E-01	2.00E + 00	0.00E + 00
AF6	2.56E + 00	1.99E-02	2.56E + 00	1.36E-15	2.56E + 00	2.70E-01	2.56E + 00	1.46E-07
AF7	1.08E + 00	2.29E-06	1.08E + 00	7.08E-02	1.08E + 00	3.47E-02	1.08E + 00	6.48E-02
AF8	9.90E + 01	3.72E + 00	9.92E + 01	4.07E + 00	1.07E + 02	6.88E + 00	9.92E + 01	7.39E + 00
AF9	2.69E + 04	1.74E-04	2.69E + 04	1.11E-11	2.69E + 04	1.11E-11	2.69E + 04	1.11E-11
AF10	5.85E + 04	2.45E + 03	5.85E + 04	5.14E + 03	5.36E + 04	1.35E + 03	5.36E + 04	1.85E + 03
AF11	2.99E + 03	5.61E-02	2.99E + 03	4.64E-13	2.99E + 03	4.64E-13	2.99E + 03	4.64E-13
AF12	2.64E + 02	3.77E-05	2.64E + 02	0.00E + 00	2.64E + 02	2.88E + 00	2.64E + 02	1.99E-02
AF13	2.35E-01	4.86E-08	2.35E-01	1.13E-16	2.35E-01	1.13E-16	2.35E-01	1.13E-16

Table 17. Performance of HEO algorithm on real world constrained CEC 2020 optimization testbed.

- Efficiently navigate multimodal landscapes, avoiding local optima.
- Achieve fast convergence on unimodal problems by intensifying the search around the global optimum.
- Adapt dynamically to different problem types, making it suitable for both unconstrained benchmarks and constrained engineering applications.

Overall, the exploration and exploitation mechanisms of HEO demonstrate a synergistic approach, inspired by nature, to tackle optimization challenges effectively. These features contribute significantly to its superior performance across a wide range of benchmark problems and real-world applications.

HEO computational complexity

To assess the scalability and efficiency of the HEO algorithm, we performed a computational complexity analysis using Big O notation. The HEO algorithm iteratively updates the positions of all candidate solutions in a population over a predefined number of iterations. The experiment in Figure 9 indicates that HEO scales linearly with respect to the problem size, population, and iterations. Empirical runtime measurements shown in Figure 9 confirm this theoretical finding, where the execution time increases proportionally with the problem size. The nearly linear relationship demonstrates HEO's suitability for large-scale optimization tasks with manageable computational overhead.

Advancing convolutional neural network training with HEO

Dataset description

The dataset used for this experiment is the MNIST dataset, a widely utilized benchmark for image classification tasks in deep learning. It consists of 70,000 grayscale images of handwritten digits (0-9), where 60,000 images are used for training and 10,000 images are reserved for testing. Each image has a size of 28×28 pixels, with pixel intensity values ranging from 0 to 255. For this experiment, the dataset was normalized by scaling pixel values to a range of 0 to 1 and reshaped to include a channel dimension for compatibility with Convolutional Neural Networks (CNNs). Additionally, labels were one-hot encoded to represent the 10 digit classes, making the dataset suitable for multi-class classification. To reduce computational load, a subset of 10,000 images was used during the hyperparameter optimization process, with an 80-20 split for training and validation. The test set of 10,000 images was kept for evaluating the final model performance after optimization. This dataset is ideal for hyperparameter tuning due to its manageable size and standardization, allowing for meaningful comparisons between different optimization algorithms.

Results and discussion

In this experiment, we optimized the hyperparameters of a Convolutional Neural Network (CNN) for the MNIST dataset using two optimization algorithms: Hare Escape Optimization (HEO) and Particle Swarm Optimization (PSO). The goal was to minimize the model's validation loss and achieve high accuracy on the test set. The convergence curves of both algorithms, shown in Figs. 10 and 11, provide insights into how each algorithm performs over iterations. Table 18 shows the optimal hyperparameters and the important matrices.

The HEO algorithm shows a consistent and stable convergence pattern, shown in Figure 10, where the validation loss decreases steadily with each iteration. After 6 iterations, HEO stabilizes around a validation loss of 0.0593, demonstrating that it can efficiently explore the hyperparameter space to find a suitable configuration without large fluctuations. Figure 11 reveals that PSO algorithm follows a similar pattern, with a rapid decrease in validation loss in the initial iterations. It converges at a slightly lower validation loss of 0.053 but exhibits more fluctuation in early iterations, suggesting less stable exploration compared to HEO. One of the key contributions of the HEO algorithm is its ability to achieve higher test accuracy. The HEO-based model achieved a test

accuracy of 0.9807, which is higher than PSO's accuracy of 0.9791. This emphasizes HEO's strength in finding hyperparameters that lead to better generalization and accuracy, which is critical in many machine learning applications where accuracy is the primary evaluation metric. HEO's convergence behavior, as seen in Figure 10, is more stable compared to PSO. It exhibits a smooth decrease in validation loss, which suggests that HEO has more consistent updates in its search for optimal hyperparameters. This stability is particularly valuable when applying HEO in sensitive models or applications where consistent improvement without large fluctuations is essential. The hyperparameter configuration selected by HEO reflects its ability to balance model complexity and regularization. For example, the dropout rate selected by HEO is 0.339703, which is significantly lower than the 0.52556 chosen by PSO. This moderate dropout rate in HEO likely contributes to its superior test accuracy by maintaining sufficient model capacity while preventing overfitting. Additionally, the batch size chosen by HEO (212) is larger than PSO's batch size (66), allowing for more stable gradient updates during training. Also, the HEO selected a more balanced number of units in the dense layer (94 units), compared to PSO's selection of 254 units. Despite using fewer units, the HEO-based model still achieved better accuracy, demonstrating that HEO is effective in avoiding over-parameterization while maintaining strong model performance.

Although PSO achieved a slightly lower final test loss (0.0593) compared to HEO (0.053), the difference is marginal. The lower test loss in PSO can be attributed to the more aggressive dropout rate and smaller batch size, which may have improved generalization in terms of loss minimization. The higher test accuracy achieved by HEO (0.9807) compared to PSO (0.9791) underscores its strength in maximizing performance. The model tuned by HEO is better suited for tasks where accuracy is prioritized over minor reductions in loss. The trade-off between loss minimization and accuracy enhancement demonstrates HEO's superior ability to find a more balanced and performant solution. One of the most significant contributions of HEO over PSO is its ability to produce a model with higher test accuracy. For applications where accuracy is the key metric, HEO demonstrates its advantage by fine-tuning hyperparameters more effectively to achieve superior generalization. HEO's more stable convergence behavior, as observed in the convergence curve, indicates its efficiency in navigating the hyperparameter space without the erratic fluctuations seen in PSO. This stability is a crucial advantage when applying HEO in real-world scenarios where robustness is necessary. Also, the HEO algorithm excels at finding a balance between regularization (through dropout) and model capacity (via the number of filters, units, and batch size). By selecting moderate hyperparameter values, HEO avoids overfitting while still achieving high accuracy, showcasing its effectiveness in model tuning. Thus, HEO's contribution to achieving higher accuracy makes it a preferable choice in practical scenarios where accuracy is more critical than slight improvements in loss. Furthermore, its stable convergence and balanced hyperparameter configuration make it a robust and reliable choice for hyperparameter optimization in deep learning models. In conclusion, HEO contributes significant advantages over PSO, particularly in terms of accuracy, stability, and the ability to balance model complexity. While PSO achieves slightly lower test loss, HEO's superior accuracy and stable optimization make it the preferred choice for many deep learning applications.

Conclusion

This study presented the Hare Escape Optimization (HEO) algorithm, which represents a novel and highly promising contribution to the field of metaheuristic optimization algorithms. Inspired by the escape strategies of hares, it leverages Levy flight dynamics and random directional changes to balance exploration and exploitation effectively within diverse search spaces. The versatility and reliability of the HEO algorithm has been evaluated across a broad spectrum of optimization challenges, including standard benchmark problems (CEC 2015 and CEC 2020) and complex real-world engineering applications. In comparative studies, HEO consistently outperformed or matched state-of-the-art algorithms in both unconstrained and constrained scenarios. It particularly excelled in solving unimodal and hybrid functions, where demonstrated its ability to converge to global optima. The robustness of HEO in handling constraints, a critical factor in engineering problems, makes it apart from many competitors that struggle with feasibility issues. Notably, HEO achieved feasible and competitive solutions in engineering applications such as spring design, welded beam design, pressure vessel optimization, and structural

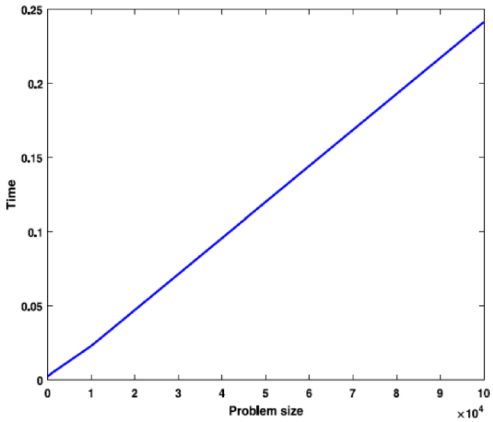


Fig. 9. Linear behavior of the time complexity of the HEO algorithm.

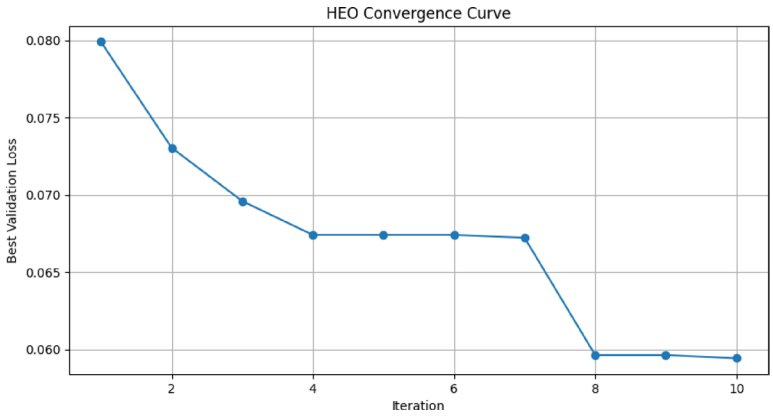


Fig. 10. Convergence curve of the HEO algorithm on optimizing CNN for MNIST dataset.

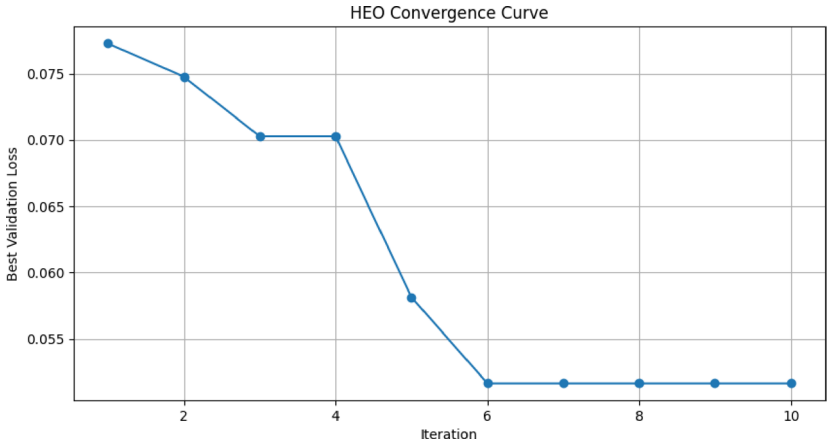


Fig. 11. Convergence curve of the HEO algorithm on optimizing CNN for MNIST dataset.

Algorithm	Best Hyperparameters		Model Test Loss	Model Test Accuracy	Final Model Test Loss	Final Model Test Accuracy
HEO	Learning Rate	0.005672	0.0898	0.9706	0.0593	0.9807
	Filters1	20				
	Filters2	103				
	Kernel Size	5				
	Units	94				
	Dropout Rate	0.339703				
	Batch Size	212				
PSO	Learning Rate	0.004600	0.0917	0.9708	0.053	0.9791
	Filters1	32				
	Filters2	101				
	Kernel Size	5				
	Units	254				
	Dropout Rate	0.525557				
	Batch Size	66				

Table 18. Best hyperparameters found by each algorithm and their respective performance metrics.

truss optimization, solidifying its potential for real-world deployment. The strengths of HEO lie in its dynamic adaptation to the search landscape, ensuring efficient exploration of the solution space while maintaining focus on promising regions. This adaptability is achieved through a carefully designed mechanism that mimics the randomness and precision of natural escape behaviors. Its relatively low variability in results underscores its reliability, making it an attractive choice for applications requiring robust and consistent performance. While

HEO's overall performance is exceptional, particularly in constrained optimization, there are opportunities for further refinement. Enhancing its capacity to navigate high-dimensional multimodal landscapes, possibly through hybridization with complementary algorithms like Differential Evolution or Genetic Algorithms, could elevate its applicability. Additionally, incorporating adaptive parameter tuning could further optimize its convergence behavior, addressing the minor variability observed in some challenging benchmarks. Despite its promising performance, this study has several limitations. First, comparative analyses relied on results reported in previous literature, limiting our ability to perform statistical significance testing such as the Wilcoxon rank-sum test. Second, some solutions in constrained problems (e.g., the welded beam design) exhibited violations, indicating that further refinement in constraint-handling techniques is necessary. Third, the computational complexity, although linear, may increase with problem dimensionality in real-time applications. Finally, the algorithm was primarily tested on single-objective optimization problems; future research should investigate its applicability to multi-objective and combinatorial problems, as well as hybridizing HEO with other techniques to improve robustness and generalization.

Data availability

The datasets used in this study are available from the corresponding author upon reasonable request.

Received: 7 May 2025; Accepted: 2 July 2025
Published online: 21 July 2025

References

1. Ghafil, H. N. & Jármai, K. Dynamic differential annealed optimization: new metaheuristic optimization algorithm for engineering applications. *Appl Soft Comput*, p. 106392, (2020).
2. Ghafil, H. N. & Jármai, K. *Optimization for Robot Modelling with MATLAB* (Springer Nature, 2020).
3. Albedran, H. & Jármai, K. Evolutionary control system of asymmetric quadcopter. *Int. Rev. Appl. Sci. Eng.* **14** (3), 374–382 (2023).
4. Alsamia, S. M., Mahmood, M. S. & Akhtarpour, A. Prediction of the contamination track in Al-Najaf city soil using numerical modelling, in *IOP Conference Series: Materials Science and Engineering*, vol. 888, no. 1, p. 12050. (2020).
5. Li, H. & Qin, S. A Hessian-based zeroing neurodynamic approach for quaternion-variable time-varying constrained optimization problems. *Neurocomputing* **564**, 126937 (2024).
6. Alsamia, S., Koch, E. & Hamadi, H. S. Comparative study of metaheuristics on optimal design of gravity retaining wall. *Pollack Period*, (2023).
7. Nagarajan, B., Svn, S. K., Selvi, M. & Thangaramya, K. A fuzzy based chicken swarm optimization algorithm for efficient fault node detection in wireless sensor networks. *Sci. Rep.* **14** (1), 27532 (2024).
8. Alsamia, S., Albedran, H. & Mahmood, M. S. Contamination depth prediction in sandy soils using fuzzy rule-based expert system. *Int. Rev. Appl. Sci. Eng.* **14** (1), 87–99 (2023).
9. Ghafil, H. N., László, K. & Jármai, K. Investigating three learning algorithms of a neural networks during inverse kinematics of robots, in Solutions for Sustainable Development, CRC, 33–40. (2019).
10. Alsamia, S. & Koch, E. Random forest regression on pullout resistance of a pile. *Pollack Period*, (2024).
11. Alsamia, S., Koch, E., EVALUATION THE BEHAVIOR OF PULLOUT FORCE AND DISPLACEMENT FOR A SINGLE & PILE: EXPERIMENTAL VALIDATION WITH PLAXIS 3D. *Kufa J. Eng.* **14** (2), 105–116 (2023).
12. Cui, E. H., Zhang, Z., Chen, C. J. & Wong, W. K. Applications of nature-inspired metaheuristic algorithms for tackling optimization problems across disciplines. *Sci. Rep.* **14** (1), 9403 (2024).
13. Alsamia, S., Albedran, H. & Jármai, K. Comparative Study of Different Metaheuristics on CEC 2020 Benchmarks, in *Vehicle and Automotive Engineering 4: Select Proceedings of the 4th VAE2022, Miskolc, Hungary*, Springer, pp. 709–719. (2022).
14. Yao, J. et al. Research on hybrid strategy particle swarm optimization algorithm and its applications. *Sci. Rep.* **14** (1), 24928 (2024).
15. Ghafil, H. N. & Jármai, K. Optimization algorithms for inverse kinematics of robots with MATLAB source code. in *Vehicle Automot. Engineering*, pp. 468–477. (2020).
16. Alsamia, S., Koch, E., Albedran, H. & Ray, R. Adaptive exploration artificial bee colony for mathematical optimization. *AI* **5** (4), 2218–2236 (2024).
17. Ghafil, H. N. & Jármai, K. Optimum dynamic analysis of a robot arm using flower pollination algorithm, (2019).
18. Wolpert, D. H. & Macready, W. G. No free lunch theorems for optimization. *IEEE Trans. Evol. Comput.* **1** (1), 67–82 (1997).
19. Deng, L. & Liu, S. A sine cosine algorithm guided by elite pool strategy for global optimization. *Appl. Soft Comput.* **164**, 111946 (2024).
20. Deng, L. & Liu, S. Snow ablation optimizer: A novel metaheuristic technique for numerical optimization and engineering design. *Expert Syst. Appl.* **225**, 120069 (2023).
21. Deng, L. & Liu, S. A multi-strategy improved slime mould algorithm for global optimization and engineering design problems. *Comput. Methods Appl. Mech. Eng.* **404**, 115764 (2023).
22. Deng, L. & Liu, S. An enhanced slime mould algorithm based on adaptive grouping technique for global optimization. *Expert Syst. Appl.* **222**, 119877 (2023).
23. Deng, L. & Liu, S. Incorporating Q-learning and gradient search scheme into JAYA algorithm for global optimization. *Artif. Intell. Rev.* **56** (Suppl 3), 3705–3748 (2023).
24. Deng, L. & Liu, S. A novel hybrid grasshopper optimization algorithm for numerical and engineering optimization problems. *Neural Process. Lett.* **55** (7), 9851–9905 (2023).
25. Deng, L. & Liu, S. Advancing photovoltaic system design: an enhanced social learning swarm optimizer with guaranteed stability. *Comput. Ind.* **164**, 104209 (2025).
26. Sörensen, K. Metaheuristics—the metaphor exposed. *Int. Trans. Oper. Res.* **22** (1), 3–18 (2015).
27. Camacho-Villalón, C. L., Dorigo, M. & Stützle, T. Exposing the grey wolf, moth-flame, whale, firefly, bat, and antlion algorithms: six misleading optimization techniques inspired by bestial metaphors. *Int. Trans. Oper. Res.* **30** (6), 2945–2971 (2023).
28. Deng, L. & Liu, S. Deficiencies of the Whale optimization algorithm and its validation method. *Expert Syst. Appl.* **237**, 121544 (2024).
29. Deng, L. & Liu, S. Metaheuristics exposed: unmasking the design pitfalls of arithmetic optimization algorithm in benchmarking. *Appl. Soft Comput.* **160**, 111696 (2024).
30. Deng, L. & Liu, S. Exposing the chimp optimization algorithm: A misleading metaheuristic technique with structural bias. *Appl. Soft Comput.* **158**, 111574 (2024).
31. Wang, X. Draco Lizard optimizer: a novel metaheuristic algorithm for global optimization problems. *Evol. Intell.* **18** (1), 1–20 (2025).

32. Hamadneh, T. et al. Magnificent Frigatebird optimization: A new Bio-Inspired metaheuristic approach for solving optimization problems. *Comput. Mater. Contin.* **80** (2), 2721–2741 (2024).
33. Wang, X. Fishing Cat optimizer: a novel metaheuristic technique. *Eng Comput*, (2025).
34. Alsamia, S., Ibrahim, D. S. & Ghafil, H. N. Optimization of drilling performance using various metaheuristics. *Pollack Period*, (2021).
35. Alsamia, S. & Koch, E. Applying clustered artificial neural networks to enhance contaminant diffusion prediction in geotechnical engineering. *Sci. Rep.* **14** (1), 28750 (2024).
36. Ghafil, H. N. & Jármai, K. Kinematic-based structural optimization of robots. *Pollack Period.* **14** (3), 213–222 (2019).
37. Rippel, O., Snoek, J. & Adams, R. P. Spectral representations for convolutional neural networks. *Adv Neural Inf. Process. Syst*, **28**, (2015).
38. Ghafil, H. N. & Jármai, K. *Research and application of industrial robot manipulators in vehicle and automotive engineering, a survey*, no. 9783319756769. (2018).
39. Winograd, S. *Arithmetic complexity of computations*, vol. 33. Siam, (1980).
40. Shen, L. et al. On efficient training of Large-Scale deep learning models. *ACM Comput. Surv*, (2024).
41. Sridhar, C. & Kanhe, A. Approximate floating point precise carry prediction adder for FIR filter applications. *Circuits Syst. Signal. Process.* **43** (10), 6487–6509 (2024).
42. Sharma, M., Heard, J., Saber, E. & Markopoulos, P. P. Convolutional Neural Network Compression via Dynamic Parameter Rank Pruning, *arXiv Prepr. arXiv2401.08014*, (2024).
43. Huang, D. et al. DWM: A decomposable Winograd method for convolution acceleration, in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, pp. 4174–4181. (2020).
44. Oyama, Y. et al. *Toward Training a Large 3d Cosmological Cnn with Hybrid Parallelization* (Lawrence Livermore National Lab. (LLNL), 2019). (United States).
45. Jia, Z., Lin, S., Qi, C. R. & Aiken, A. Exploring Hidden Dimensions in Parallelizing Convolutional Neural Networks., in *ICML*, vol. 2279, p. 2288. (2018).
46. Szegedy, C. et al. Going deeper with convolutions, in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1–9. (2015).
47. Huang, G., Liu, S., Van der Maaten, L. & Weinberger, K. Q. Condensenet: An efficient densenet using learned group convolutions, in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2752–2761. (2018).
48. Tai, C., Xiao, T., Zhang, Y. & Wang, X. Convolutional neural networks with low-rank regularization, *arXiv Prepr. arXiv1511.06067*, (2015).
49. Huang, C. T. et al. ecnn: A block-based and highly-parallel cnn accelerator for edge inference, in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 182–195. (2019).
50. Song, L. et al. Hypar: Towards hybrid parallelism for deep learning accelerator array, in *2019 IEEE international symposium on high performance computer architecture (HPCA)*, pp. 56–68. (2019).
51. Earth, B. & Hare, W. P. H. A. *Ytube Video*, (2017). https://www.youtube.com/watch?v=gGludGaPKag (accessed Feb. 07, 2025).
52. Liang, J. J., Qu, B. Y., Suganthan, P. N. & Chen, Q. Problem definitions and evaluation criteria for the CEC 2015 competition on learning-based real-parameter single objective optimization, *Tech. Report201411A, Comput. Intell. Lab. Zhengzhou Univ. Zhengzhou China Tech. Report, Nanyang Technol. Univ. Singapore*, vol. 29, pp. 625–640, (2014).
53. Aydilek, I. B. A hybrid firefly and particle swarm optimization algorithm for computationally expensive numerical problems. *Appl. Soft Comput.* **66**, 232–249 (2018).
54. Eberhart, R. & Kennedy, J. Particle swarm optimization, in *Proceedings of the IEEE international conference on neural networks*, vol. 4, pp. 1942–1948. (1995).
55. Fister, I., Fister Jr, I., Yang, X. S. & Brest, J. A comprehensive review of firefly algorithms. *Swarm Evol. Comput.* **13**, 34–46 (2013).
56. Kora, P. & Krishna, K. S. R. Hybrid firefly and particle swarm optimization algorithm for the detection of bundle branch block. *Int. J. Cardiovasc. Acad.* **2** (1), 44–48 (2016).
57. Arunachalam, S., AgnesBhomila, T. & Ramesh Babu, M. Hybrid particle swarm optimization algorithm and firefly algorithm based combined economic and emission dispatch including valve point effect, in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 8947, pp. 647–660, (2015). https://doi.org/10.1007/978-3-319-20294-5_56
58. Mirjalili, S. & Optimizer, G. W. *Website*, (2019). http://www.alimirjalili.com/GWO.html (accessed Oct. 20, 2019).
59. Surjanovic, D. S.B., Test Functions and Datasets, *Virtual Library of Simulation Experiments*, (2013). http://www.sfu.ca/~ssurjano (accessed Aug. 17, 2019).
60. Jain, M., Singh, V. & Rani, A. A novel nature-inspired algorithm for optimization: squirrel search algorithm. *Swarm Evol. Comput.* **44**, 148–175 (2019).
61. Chen, Y. et al. Particle swarm optimizer with crossover operation. *Eng. Appl. Artif. Intell.* **70**, 159–169 (2018).
62. Yang, X. S. A new metaheuristic bat-inspired algorithm, in Nature Inspired Cooperative Strategies for Optimization (NICSO 2010), Springer, 65–74. (2010).
63. Yang, X. S. & He, X. Firefly algorithm: recent advances and applications. *Int. J. Swarm Intell.* **1** (1), 36. https://doi.org/10.1504/ijsi.2013.055801 (2013).
64. Mirjalili, S., Mirjalili, S. M. & Hatamlou, A. Multi-verse optimizer: a nature-inspired algorithm for global optimization. *Neural Comput. Appl.* **27** (2), 495–513 (2016).
65. Gandomi, A. H. & Alavi, A. H. Krill herd: a new bio-inspired optimization algorithm. *Commun. Nonlinear Sci. Numer. Simul.* **17** (12), 4831–4845 (2012).
66. Mirjalili, S., Mirjalili, S. M. & Lewis, A. Grey Wolf optimizer. *Adv. Eng. Softw.* **69**, 46–61 (2014).
67. Kohli, M. & Arora, S. Chaotic grey Wolf optimization algorithm for constrained optimization problems. *J. Comput. Des. Eng.* **5** (4), 458–472 (2018).
68. Askarzadeh, A. A novel metaheuristic method for solving constrained engineering optimization problems: crow search algorithm. *Comput. Struct.* **169**, 1–12 (2016).
69. Coello, C. A. An updated survey of GA-based multiobjective optimization techniques. *ACM Comput. Surv.* **32** (2), 109–143 (2000).
70. Coello, C. A. C. & Montes, E. M. Constraint-handling in genetic algorithms through the use of dominance-based tournament selection. *Adv. Eng. Inf.* **16** (3), 193–203 (2002).
71. He, Q. & Wang, L. An effective co-evolutionary particle swarm optimization for constrained engineering design problems. *Eng. Appl. Artif. Intell.* **20** (1), 89–99 (2007).
72. Yang, J. & Jin, Y. Hierarchy Particle Swarm Optimization Algorithm (HPSO) and Its Application in Multi-Objective Operation of Hydropower Stations, in *2011 3rd International Workshop on Intelligent Systems and Applications*, pp. 1–4. (2011).
73. Coelho, L. S. Gaussian quantum-behaved particle swarm optimization approaches for constrained engineering design problems. *Expert Syst. Appl.* **37** (2), 1676–1683 (2010).
74. Zhang, M., Luo, W. & Wang, X. Differential evolution with dynamic stochastic selection for constrained optimization. *Inf. Sci. (Ny).* **178** (15), 3043–3074 (2008).
75. Liu, H., Cai, Z. & Wang, Y. Hybridizing particle swarm optimization with differential evolution for constrained numerical and engineering optimization. *Appl. Soft Comput.* **10** (2), 629–640 (2010).

76. Ray, T. & Liew, K. M. Society and civilization: an optimization algorithm based on the simulation of social behavior. *IEEE Trans. Evol. Comput.* **7** (4), 386–396 (2003).
77. Parsopoulos, K. E. & Vrahatis, M. N. Unified particle swarm optimization for solving constrained engineering optimization problems, in *International conference on natural computation*, pp. 582–591. (2005).
78. Mezura-Montes, E. & Coello, C. A. C. A simple multimembered evolution strategy to solve constrained optimization problems. *IEEE Trans. Evol. Comput.* **9** (1), 1–17 (2005).
79. Akay, B. & Karaboga, D. Artificial bee colony algorithm for large-scale problems and engineering design optimization. *J. Intell. Manuf.* **23** (4), 1001–1014 (2012).
80. Rao, R. V., Savsani, V. J. & Vakharia, D. P. Teaching–learning-based optimization: a novel method for constrained mechanical design optimization problems. *Comput. Des.* **43** (3), 303–315 (2011).
81. Sadollah, A., Bahreininejad, A., Eskandar, H. & Hamdi, M. Mine blast algorithm: A new population based algorithm for solving constrained engineering optimization problems. *Appl. Soft Comput.* **13** (5), 2592–2612 (2013).
82. Rashedi, E., Nezamabadi-Pour, H. & Saryazdi, S. GSA: a gravitational search algorithm. *Inf. Sci. (Ny)*. **179** (13), 2232–2248 (2009).
83. Coello Coello, C. A. Constraint-handling using an evolutionary multiobjective optimization technique. *Civ. Eng. Syst.* **17** (4), 319–346 (2000).
84. Deb, K. Optimal design of a welded beam via genetic algorithms. *AIAA J.* **29** (11), 2013–2015 (1991).
85. Deb, K. An efficient constraint handling method for genetic algorithms. *Comput. Methods Appl. Mech. Eng.* **186**, 2–4 (2000).
86. Mahdavi, M., Fesanghary, M. & Damangir, E. An improved harmony search algorithm for solving optimization problems. *Appl. Math. Comput.* **188** (2), 1567–1579 (2007).
87. Ragsdell, K. M. & Phillips, D. T. Optimal design of a class of welded structures using geometric programming. *J. Eng. Ind.* **98** (3), 1021–1025 (1976).
88. Coello, C. A. C. Use of a self-adaptive penalty approach for engineering optimization problems. *Comput. Ind.* **41** (2), 113–127 (2000).
89. Deb, K. GeneAS: A robust optimal design technique for mechanical component design, in Evolutionary Algorithms in Engineering Applications, Springer, 497–514. (1997).
90. Mezura-Montes, E. & Coello, C. A. C. An empirical study about the usefulness of evolution strategies to solve constrained optimization problems. *Int. J. Gen. Syst.* **37** (4), 443–473 (2008).
91. Huang, F., Wang, L. & He, Q. An effective co-evolutionary differential evolution for constrained optimization. *Appl. Math. Comput.* **186** (1), 340–356 (2007).
92. Kaveh, A. & Talatahari, S. An improved ant colony optimization for constrained engineering design problems. *Eng. Comput.* **27** (1), 155–182 (2010).
93. Kannan, B. K. & Kramer, S. N. An augmented Lagrange multiplier based method for mixed integer discrete continuous optimization and its applications to mechanical design. *J. Mech. Des.* **116** (2), 405–411 (1994).
94. Sandgren, E. Nonlinear integer and discrete programming in mechanical design, in *Proceeding of the ASME Design Technology Conference*, pp. 95–105. (1988).
95. Camp, C. V. & Farshchin, M. Design of space trusses using modified teaching–learning based optimization. *Eng. Struct.* **62**, 87–97 (2014).
96. Camp, C., Pezeshk, S. & Cao, G. Optimized design of two-dimensional structures using a genetic algorithm. *J. Struct. Eng.* **124** (5), 551–559 (1998).
97. Schutte, J. F. & Groenwold, A. A. Sizing design of truss structures using particle swarms. *Struct. Multidiscip Optim.* **25**, 261–269 (2003).
98. Li, L. J., Huang, Z. B., Liu, F. & Wu, Q. H. A heuristic particle swarm optimizer for optimization of pin connected structures. *Comput. Struct.* **85**, 7–8 (2007).
99. Lee, K. S. & Geem, Z. W. A new structural optimization method based on the harmony search algorithm. *Comput. Struct.* **82**, 9–10 (2004).
100. Degertekin, S. O. Improved harmony search algorithms for sizing optimization of truss structures. *Comput. Struct.* **92**, 229–241 (2012).
101. Sonmez, M. Artificial bee colony algorithm for optimization of truss structures. *Appl. Soft Comput.* **11** (2), 2406–2418 (2011).

Author contributions

Doaa Alsmaee designed the methodology and developed the experimental framework. Doaa Alsmaee implemented the algorithms and conducted the experiments as well as writing the article. Reza Ramezani analyzed the results and contributed to the interpretation. Doaa Alsmaee wrote the initial draft of the manuscript, while Reza Ramezani revised and supervised the wrork. Both authors reviewed and approved the final version of the manuscript.

Declarations

Competing interests

The authors declare no competing interests.

Data availablity

The datasets used in this study are available from the corresponding author upon reasonable request.

Additional information

Correspondence and requests for materials should be addressed to R.R.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025